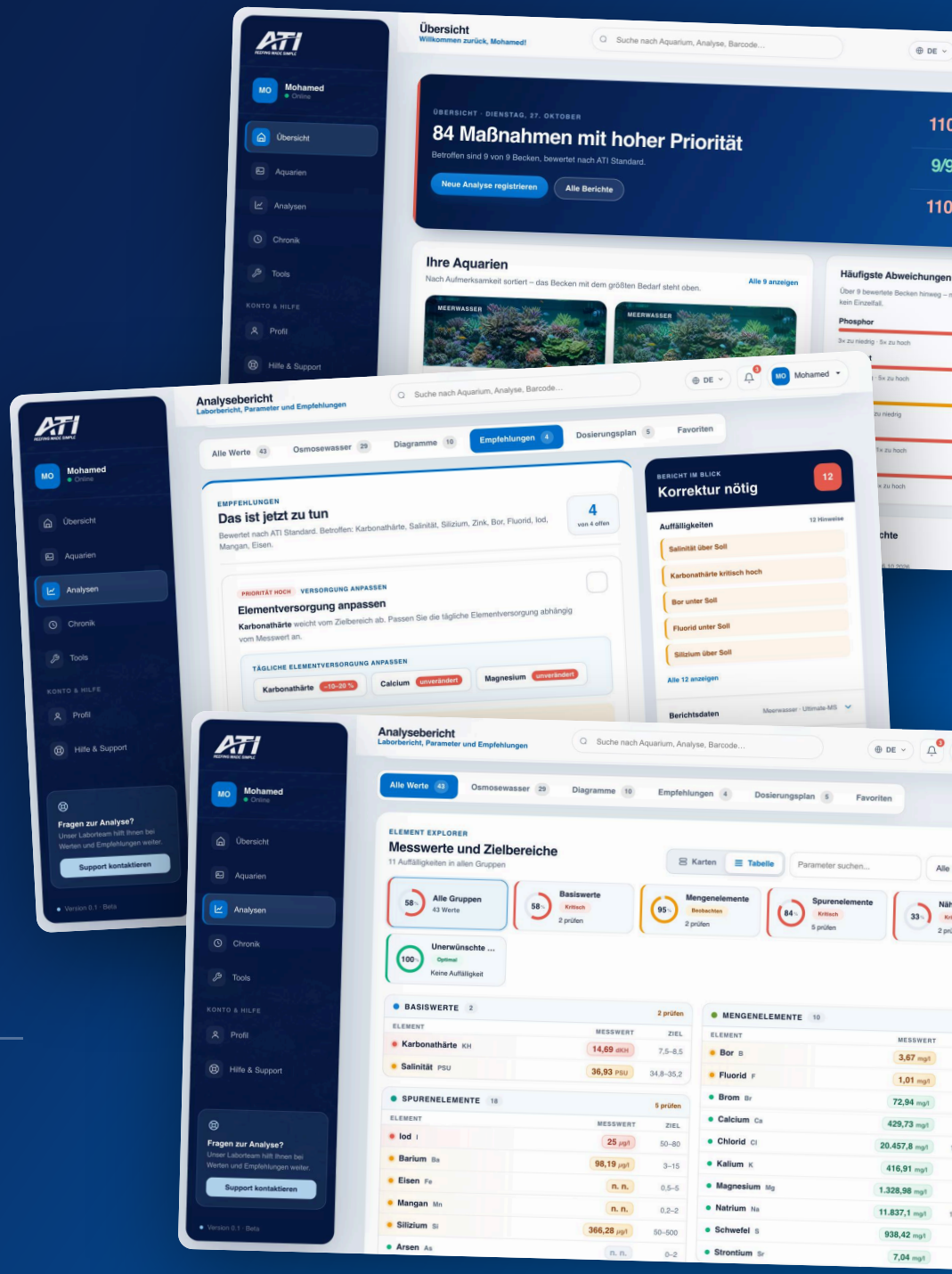
REEF PILOT · ATI / NYOS CUSTOMER PORTAL

Backend Playbook

The app as it exists today, screen by screen – how its data moves, what it needs from a server and what it never will – and a ranked list of backend tasks, each with its goal, the steps and when it counts as done.



22	17	~30	14	5–7
screens and tabs	browser storage keys to replace	API endpoints	tasks, easiest first	weeks for one developer

State of branch feat/aquarium-wizard-gauge-chronik

START HERE

How to use this playbook

Four parts. Read parts 1–3 once to understand the app; work through part 4 task by task. Every function and file name is written exactly as in the code, so a full-text search finds it.

PART 1 · P. 3

The app

A map of all screens, then every screen with a real screenshot: what it shows, what it reads and writes, which functions it calls today and which endpoint replaces them.

PART 2 · P. 17

How data moves

Where data lives today (only in the browser), the problems that causes, and the same flows with a backend – as diagrams, including one request traced end to end.

PART 3 · P. 20

Backend or not

A matrix of screens against backend capabilities, the calculation engine that stays in the browser, and the data model the server stores.

PART 4 · P. 23

Roadmap & playbooks

14 tasks ranked from easiest to hardest with dependencies, then each task with its goal, endpoints, steps, definition of done and pitfalls – what to build, not how to write it.

LEGEND · WHAT A SCREEN NEEDS FROM THE BACKEND

No backend	Static content or pure calculation. Works unchanged.
Reads	Shows data that must come from the server.
Reads & writes	Also creates or changes data – needs validation on the server.
Admin	Only for roles `admin` / `subadmin`; changes what every customer sees.
Lab	Data that comes from the ATI laboratory, not from a user.

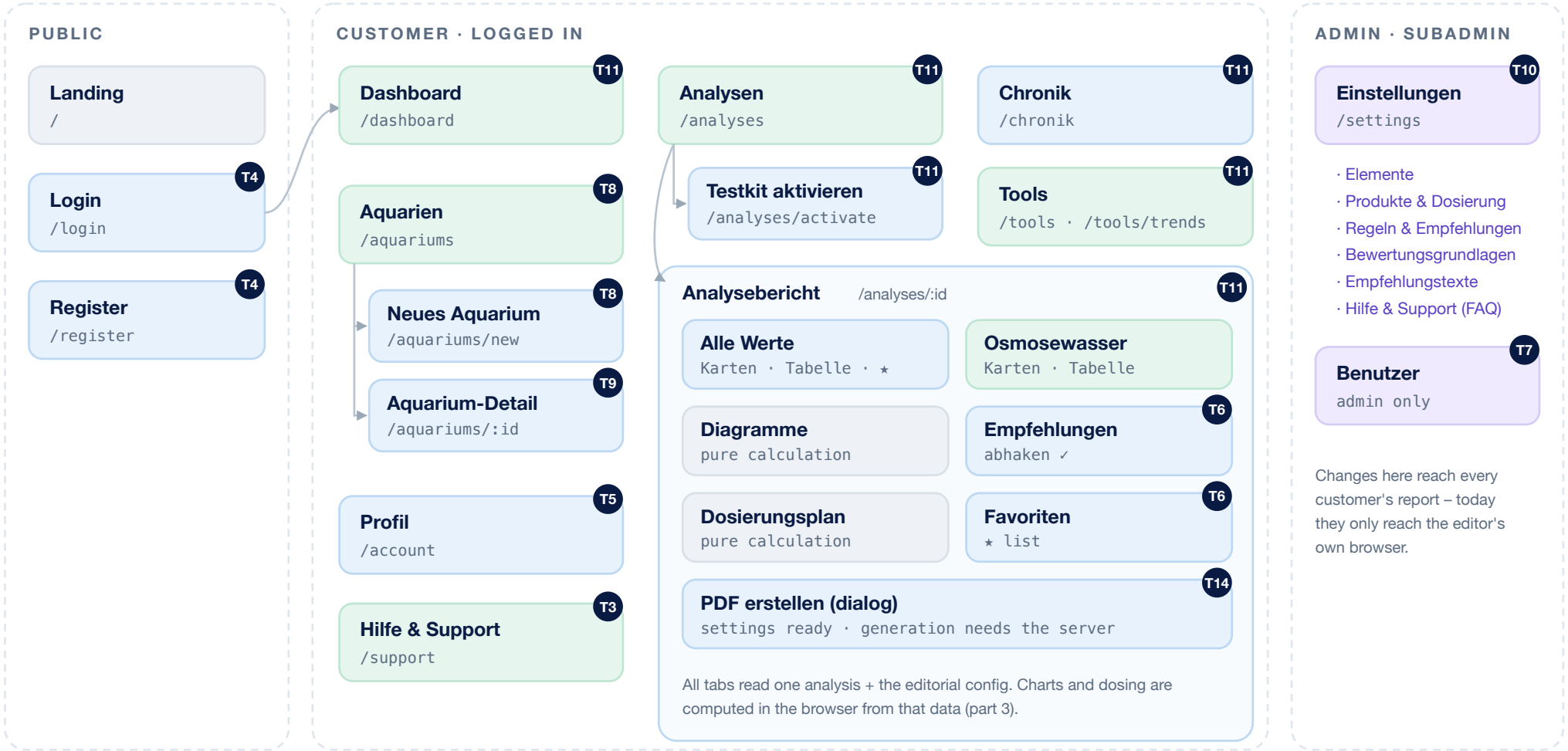
CONVENTIONS

- **T4** refers to a task in part 4.
- reads · writes · stays in browser chips on the screen pages.
- GET · POST · PUT · PATCH · DEL all paths start with `/api`.
- German texts in quotes are what the UI shows today – keep them for server error messages.
- Recommendations (stack, libraries, schema choices) are marked as such; everything else describes existing code.

PART 1 · THE APP

Every screen at a glance

22 screens and report tabs in three areas. The colour says how much backend each one needs; the number in the corner is the task that unlocks it.



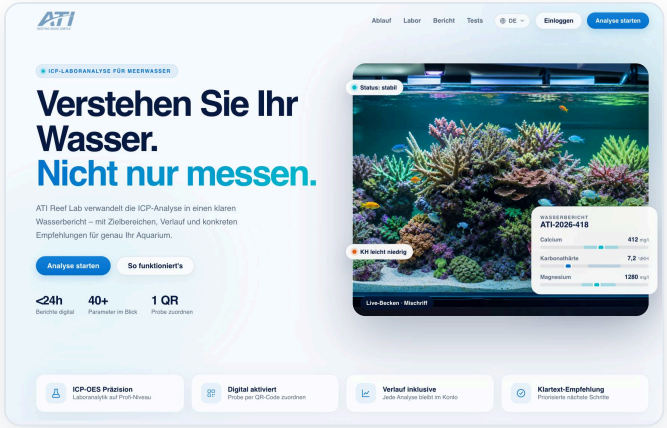
No backend · Reads · Reads & writes · Admin

SCREENS 1–3 · PUBLIC

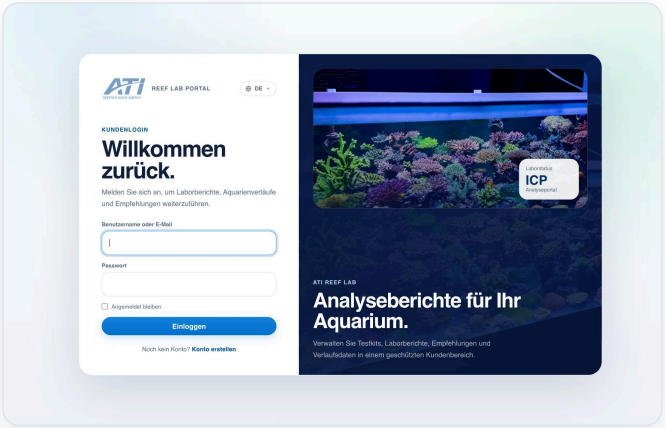
Landing, login and registration

The only screens a visitor sees without an account. The landing page is static; login and registration are the first thing that must talk to the server.

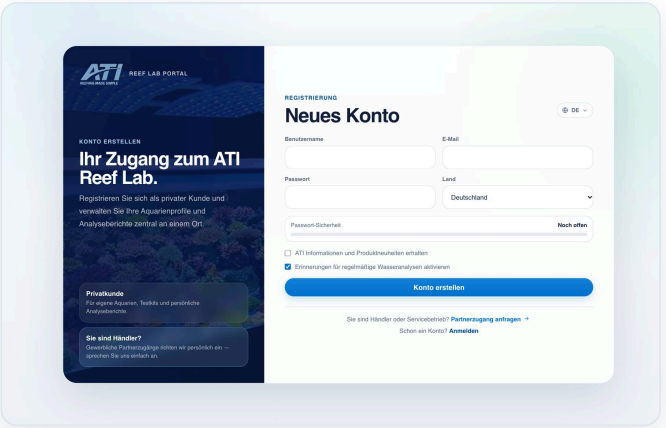
T4



Landing · `/`



Login · `/login`



Registrierung · `/register`

No backend

Marketing page built from `atiLandingContent.js`. The header only asks the auth store whether someone is logged in to show "Dashboard" instead of "Anmelden".

DATA

static texts · auth.isLoggedIn

Reads & writes

Username **or** e-mail plus password. Wrong data shows „Benutzername/E-Mail oder Passwort ist falsch." Demo accounts (`kunde` , `admin` … / `demo123`) are seeded into every browser today.

`auth.login()`	`POST /auth/login`
`localStore.loadSession()`	`GET /auth/me`

Reads & writes

Username, e-mail, password, country. Today the password is stored **in plain text** in `localStorage` and the "token" is `local-<userId>` .

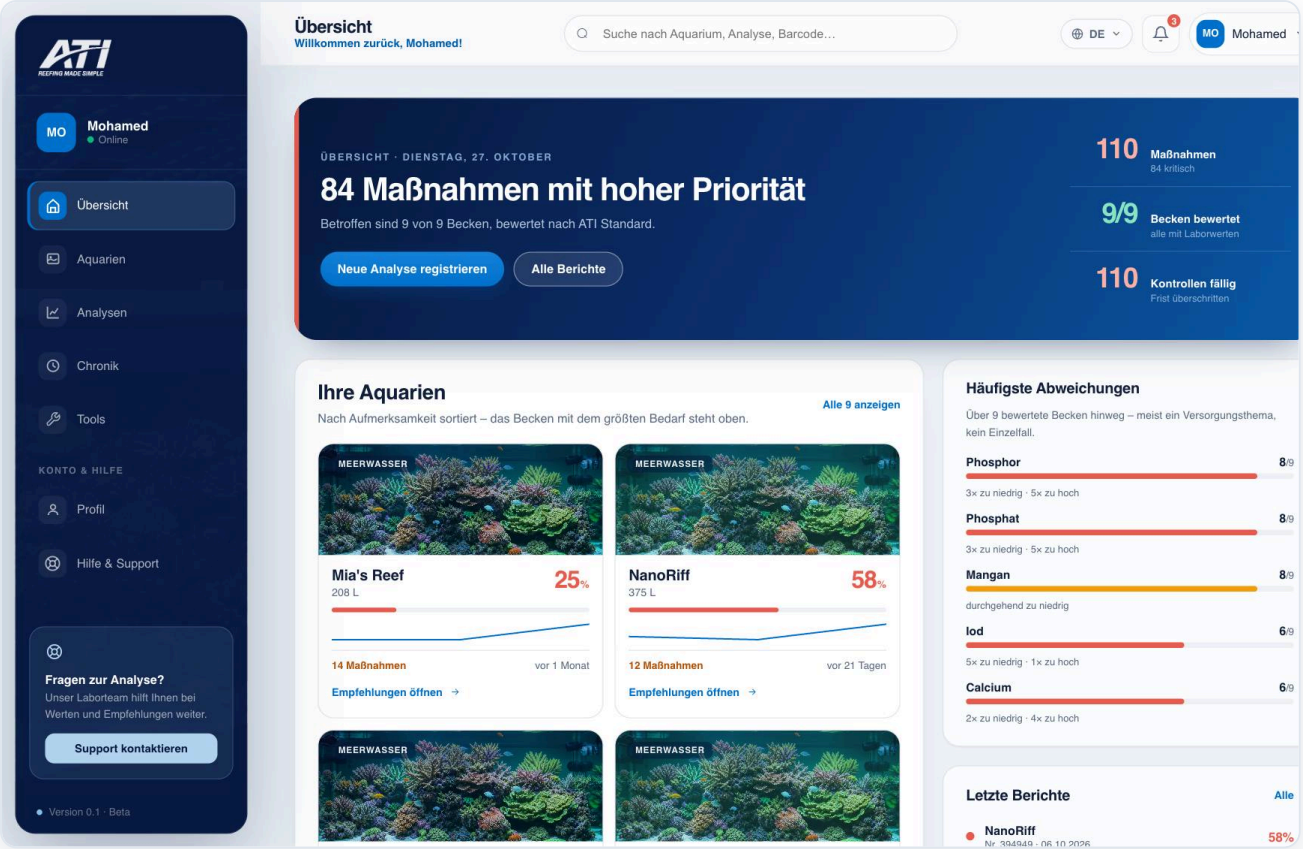
`auth.register()`	`POST /auth/register`
`auth.logout()`	`POST /auth/logout`

SCREEN 4 · /DASHBOARD

Dashboard

The start page after login. Everything on it is calculated from two lists – the user's aquariums and analyses – plus the editorial config.

● Reads T8 T11



Full dashboard of the demo customer: 9 tanks, 110 actions, most frequent deviations, latest reports. New users (`kind: 'new'`) get a setup dashboard instead.

WHAT HAPPENS

- Hero counts actions and overdue checks across all tanks.
- Tank cards are sorted by need, with score and a small trend line.
- "Häufigste Abweichungen" counts the same element across tanks.
- No input – nothing is written.

DATA

aquariums · analyses + parameters · config: scales, content, dosing · recommendations computed here

CALLS TODAY → FUTURE

`aquariums.load()`	`GET /aquariums`
`analyses.load()`	`GET /analyses`
`loadEvaluationScales()`	`GET /config`
`buildDirectRecommendations()`	`stays in browser`
`buildDosingPlan()`	`stays in browser`

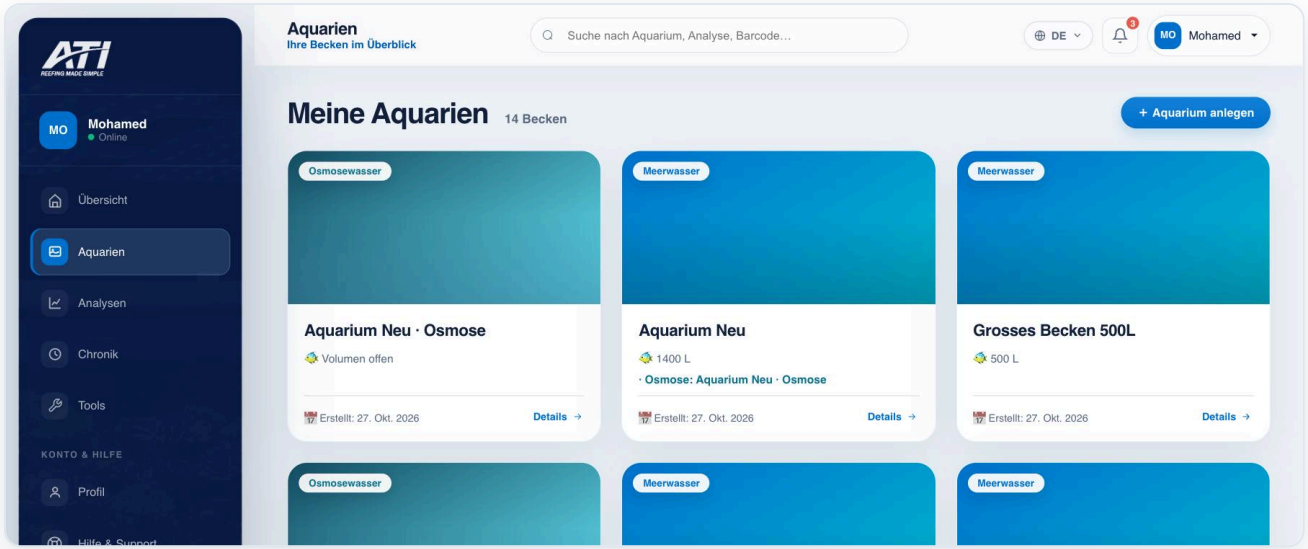
Derive, don't store: the frontend reads `user.kind` (`new` / `full`) to pick the dashboard. The server can return `full` as soon as the user has an aquarium.

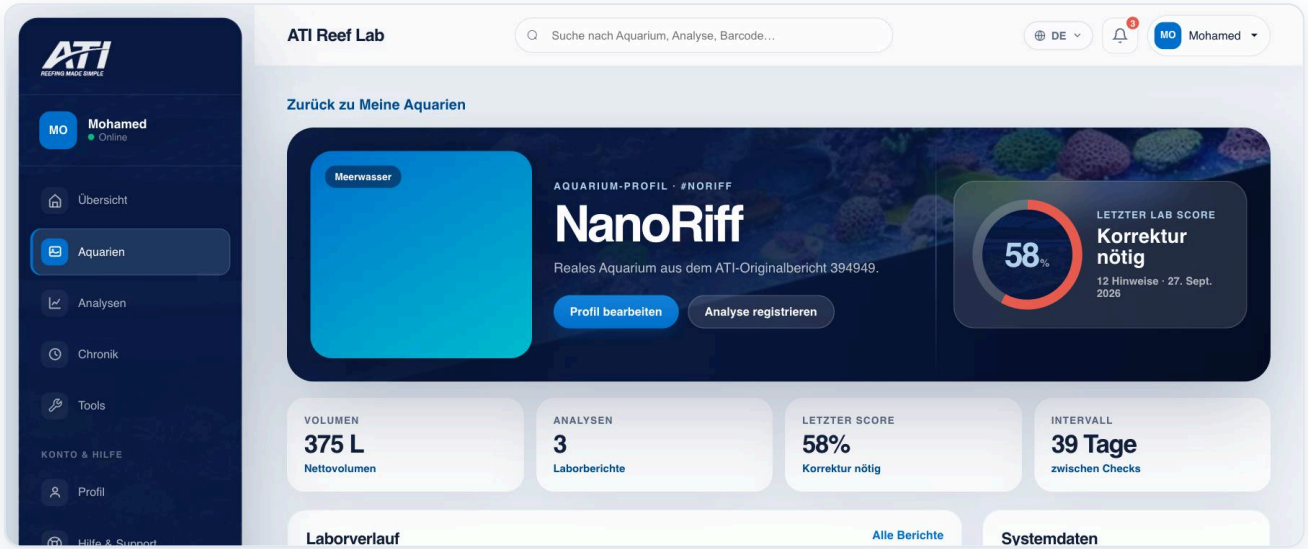
SCREENS 5–6 · /AQUARIUMS · /AQUARIUMS/:ID

Aquariums and aquarium detail

The list of the customer's tanks and one tank's profile. Osmosis water sources are aquariums too (water type „Osmosewasser“) and are linked to the tanks they feed.

● Reads & writes T8 T9





AQUARIEN · LIST

Cards with photo or illustration (`image_theme`), water type, volume. Reads only.

AQUARIUM-DETAIL

- Profile hero with last lab score and the tank's report history chart.
- "Systemdaten": water type, tank type, dimensions, target mode, stocking, equipment.
- "Profil bearbeiten" edits everything incl. photo, supply system and osmosis source; delete is here too.

DATA

aquariums · analyses of this tank · update profile · upload photo · delete

CALLS TODAY → FUTURE

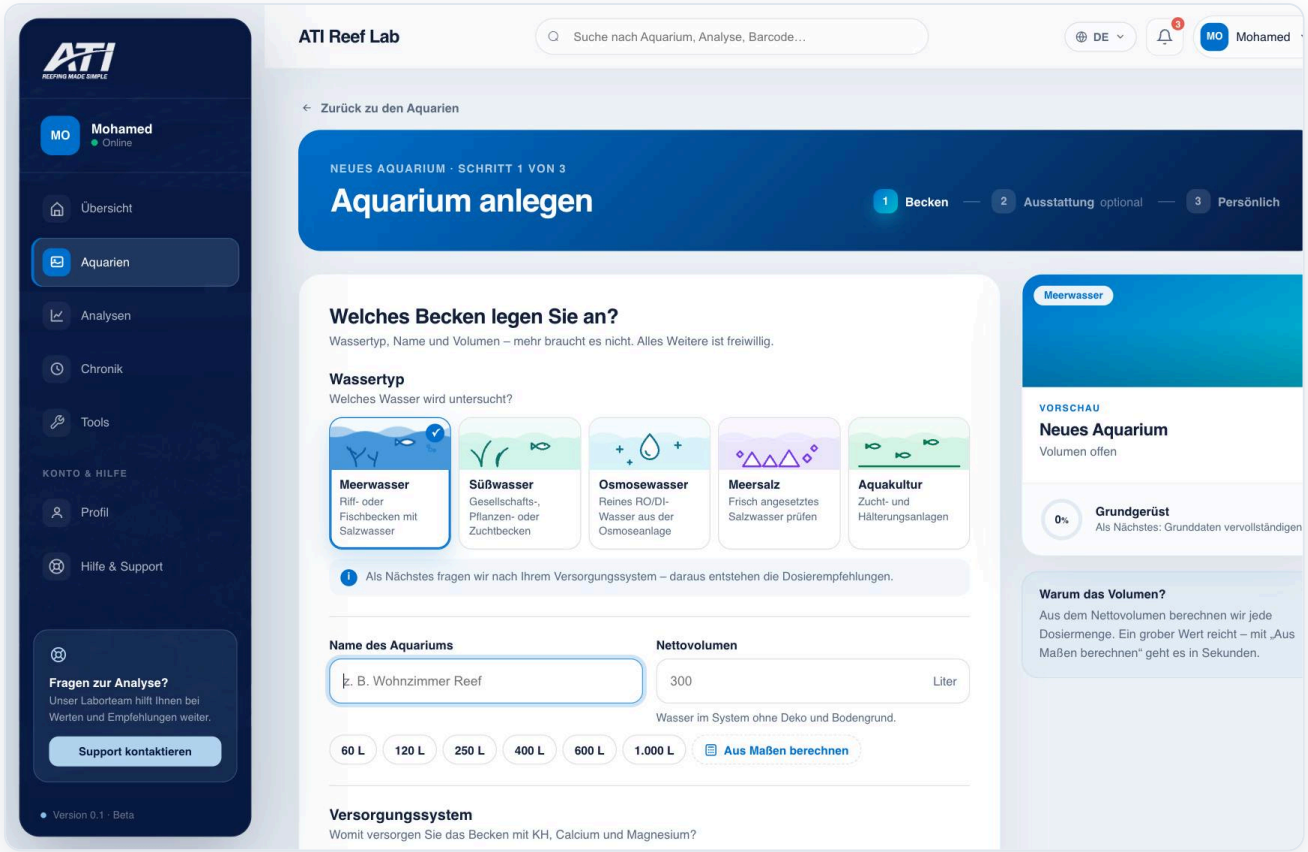
`aquariums.load()`	`GET /aquariums`
`aquariums.byId(id)`	`GET /aquariums/:id`
`aquariums.update(id, patch)`	`PATCH /aquariums/:id`
`aquariums.remove(id)`	`DEL /aquariums/:id`
`fileToResizedDataUrl()`	`POST /aquariums/:id/image`

SCREEN 7 · /AQUARIUMS/NEW

New aquarium wizard

A step-by-step form: water type, volume (with calculator), quick tiles for equipment, supply system (ATI Essentials, NYOS ION-B, other), osmosis source and evaluation scale.

● Reads & writes T8



Each water type shows only its own fields (`waterTypeFields.js → WATER_TYPE_FIELDSETS`). The supply system decides how the report adjusts daily dosing (V2.2 rules).

WHAT HAPPENS

- The draft is saved locally after every step, so a reload does not lose input.
- "Speichern" creates the tank and shows a success step.
- The osmosis picker lists the user's tanks of type „Osmosewasser“.
- The scale picker lists the evaluation scales from the config.

DATA

create aquarium · osmosis sources · evaluation scales · draft (aquariumWizard.saveDraft) · volume calculator

CALLS TODAY → FUTURE

`aquariums.create(data)`	`POST /aquariums`
`osmosisSources(ownerId)`	`GET /aquariums`
`loadEvaluationScales()`	`GET /config`
`saveDraft / loadDraft`	`stays in browser`

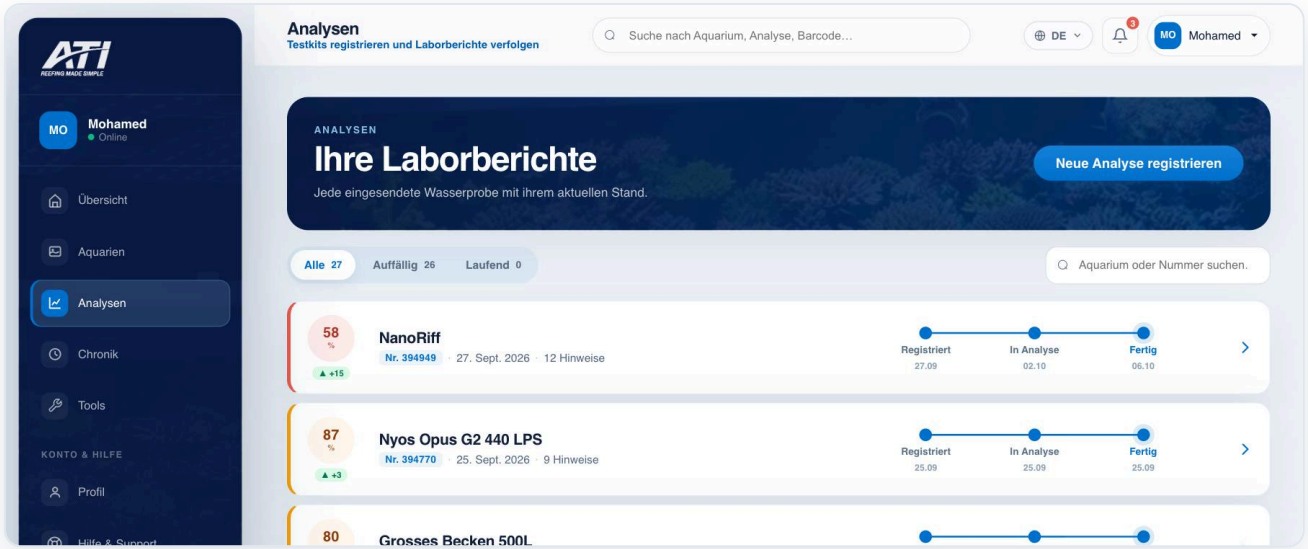
Sync today, async tomorrow: the view does `created.value = aquariums.create(data)` and uses the result immediately. It must `await` once the store calls the API (T8).

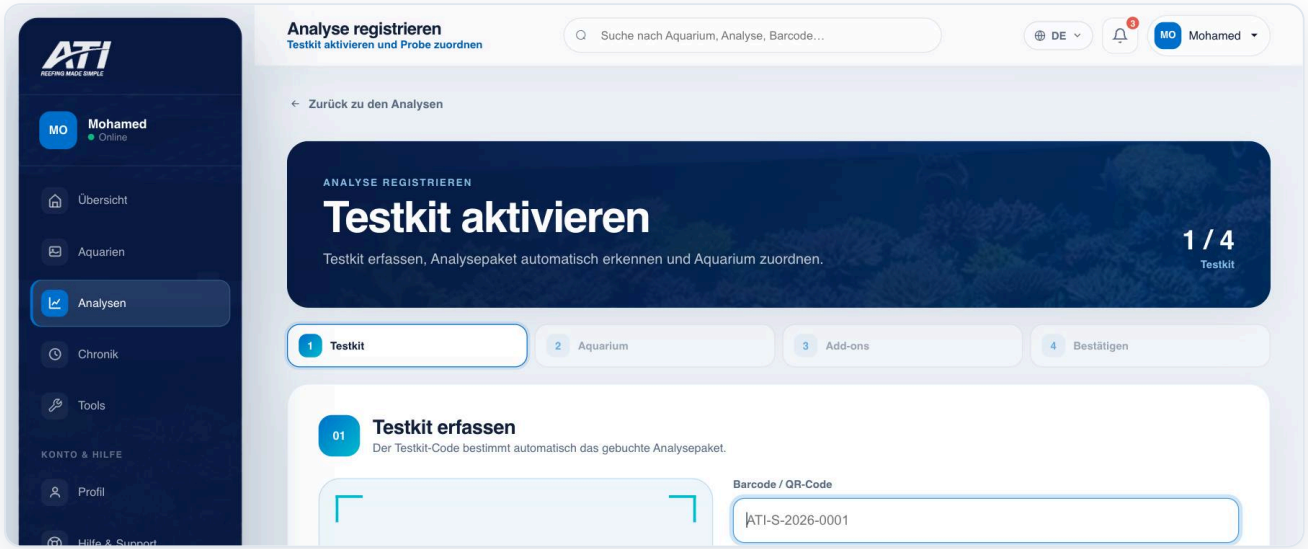
SCREENS 8–9 · /ANALYSES · /ANALYSES/ACTIVATE

Analyses and kit activation

Every water sample the customer sent, with its lab status – and the 4-step wizard that registers a new test kit before it goes to the lab.

● Reads & writes T11 T12





ANALYSEN · LIST

Reports newest first, each with the lab timeline **Registriert** → **Eingegangen** → **In Analyse** → **Fertig**. Finished ones open the report.

TESTKIT AKTIVIEREN · 4 STEPS

1. **Testkit:** barcode typed or scanned; the prefix picks the package – `ATI-S` Standard, `ATI-P` Pro, `ATI-U` Ultimate-MS.
2. **Aquarium:** which tank, plus an osmosis source for Pro/Ultimate.
3. **Add-ons:** reason (Routine, Algen, Cyanos, STN, RTN) and extras (SAK254, Express, Fachberater).
4. **Bestätigen** → status `registered`.

CALLS TODAY → FUTURE

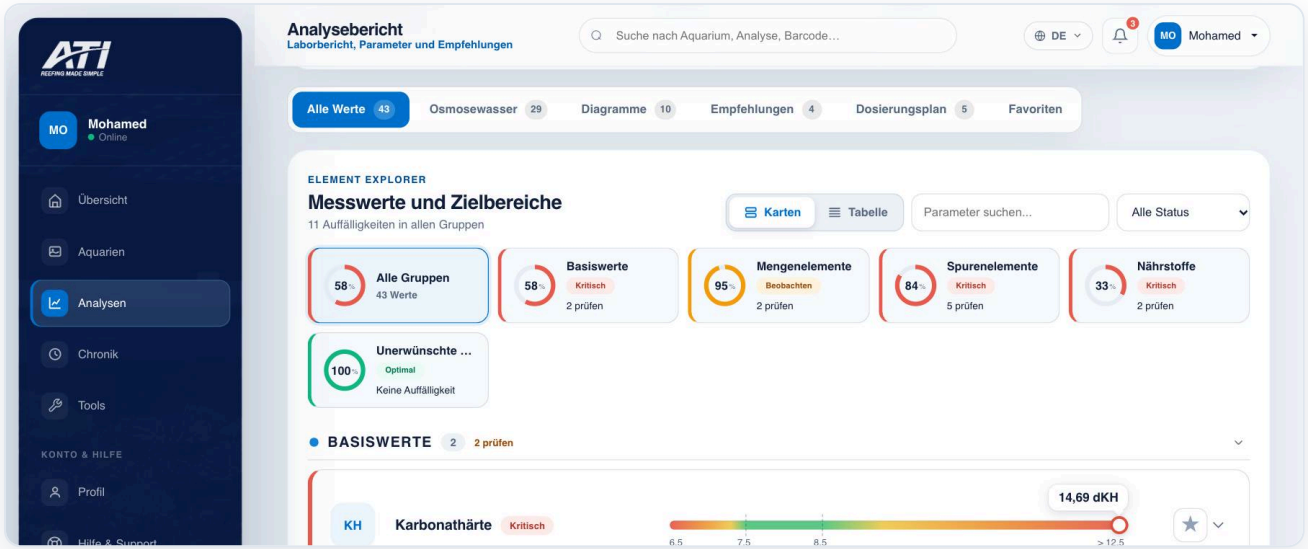
`analyses.load()`	`GET /analyses`
`packageForBarcode()`	`GET /kits/:barcode`
`analyses.activate(data)`	`POST /analyses`
– (no lab yet)	`POST /lab/…`

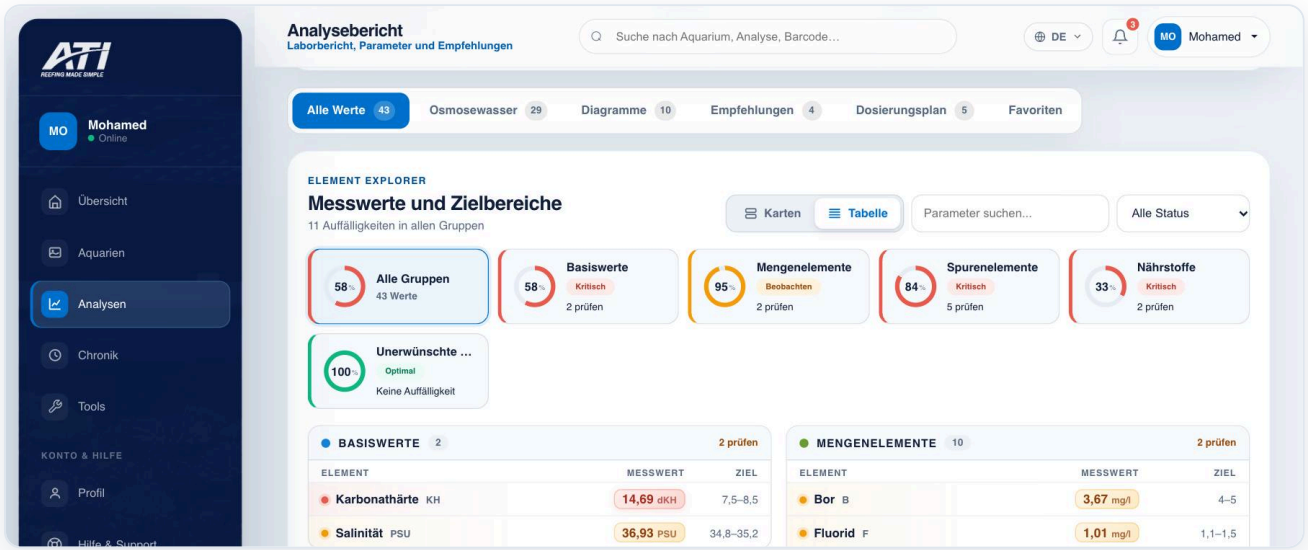
SCREEN 10 · /ANALYSES/:ID · TAB „ALLE WERTE“

The report – all values

The heart of the app. One finished analysis: 43 measured elements in five groups, each on a scale with its target range – as cards with details, or as a compact table.

● Reads & writes T11 T6





WHAT HAPPENS

- Group filters, search and a status filter narrow the list.
- Opening a card shows knowledge texts (from the editorial config) and the element's history across earlier reports.
- ★ marks a favourite element – the only write on this tab.
- „Karten | Tabelle“ switches the view; the choice is a browser preference.

DATA

analysis.parameters · parameter history · config: content, scales · favourite ★ · cards / table choice

CALLS TODAY → FUTURE

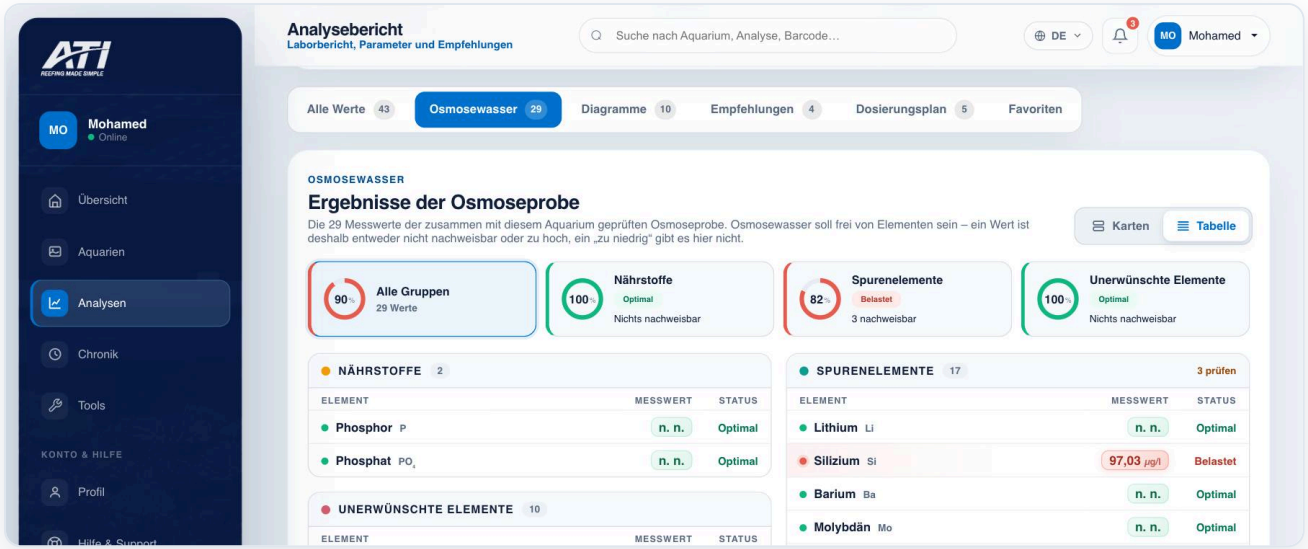
`analyses.load() + find(id)`	`GET /analyses/:id`
`loadAnalysisContent()`	`GET /config`
`analyses.toggleFavorite(key)`	`PUT /me/documents/…`
`evaluateAnalysis()`	stays in browser
`reef-pilot:values-view`	stays in browser

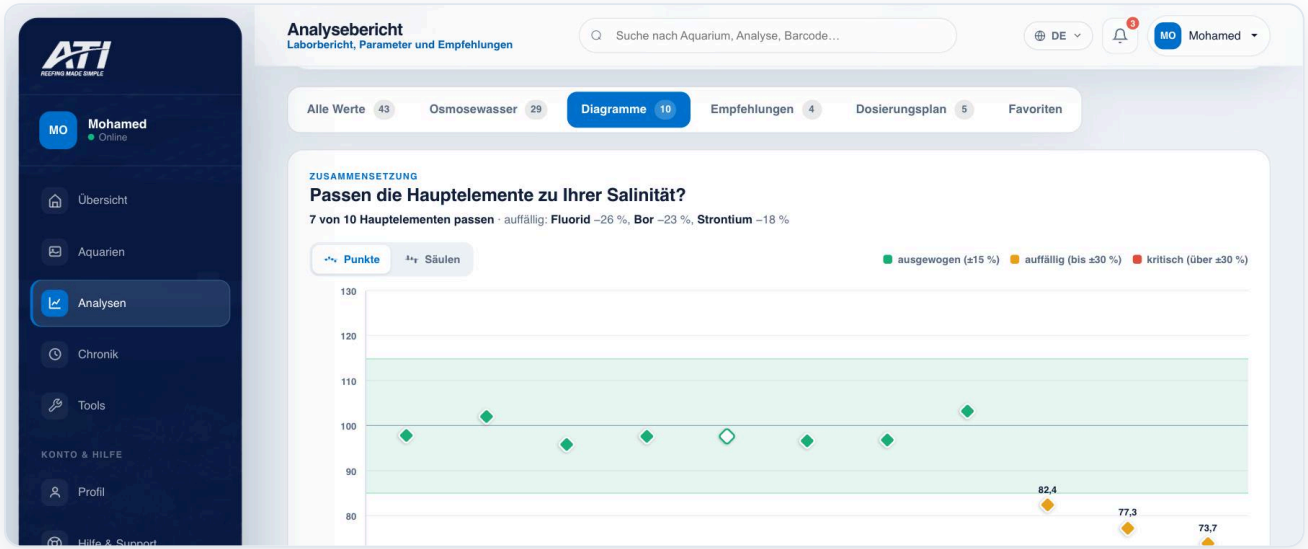
SCREENS 11–12 · TABS „OSMOSEWASSER“ · „DIAGRAMME“

Osmosis water and charts

Two read-only views of the same analysis. The osmosis tab shows the 29 values of the osmosis sample; the charts compare the water's composition with natural sea water.

Reads T11





OSMOSEWASSER

Osmosis water should contain nothing: a value is either **n. n.** (good) or detectable (bad). Same cards / table switch as „Alle Werte“.

`analysis.osmosisParameters`

DIAGRAMME · PURE CALCULATION

- **Zusammensetzung:** every element as a share of what fits the measured salinity – green band ±15 %.
- **Verhältnisse** and **Wachstumsfaktoren:** pair gauges (Mg : Ca, Ca : Sr …).

Everything comes from `waterBalance.js` – no request beyond the analysis itself.

`analysis.parameters` `waterBalance()`

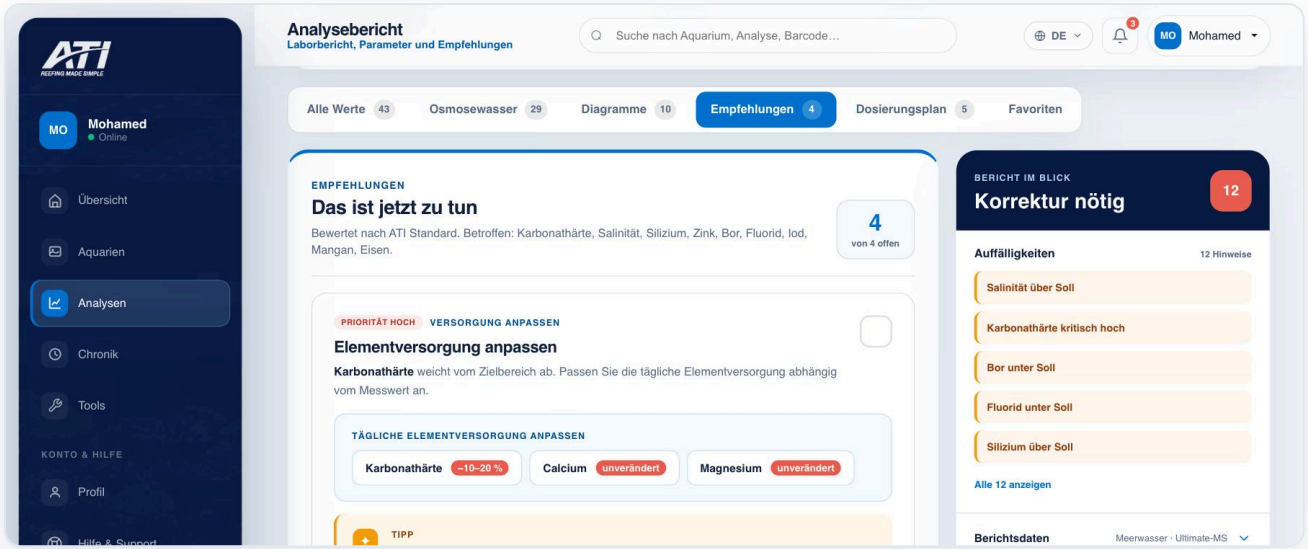
Nothing to build here once T11 delivers `parameters` and `osmosisParameters` in the shape described on page 22.

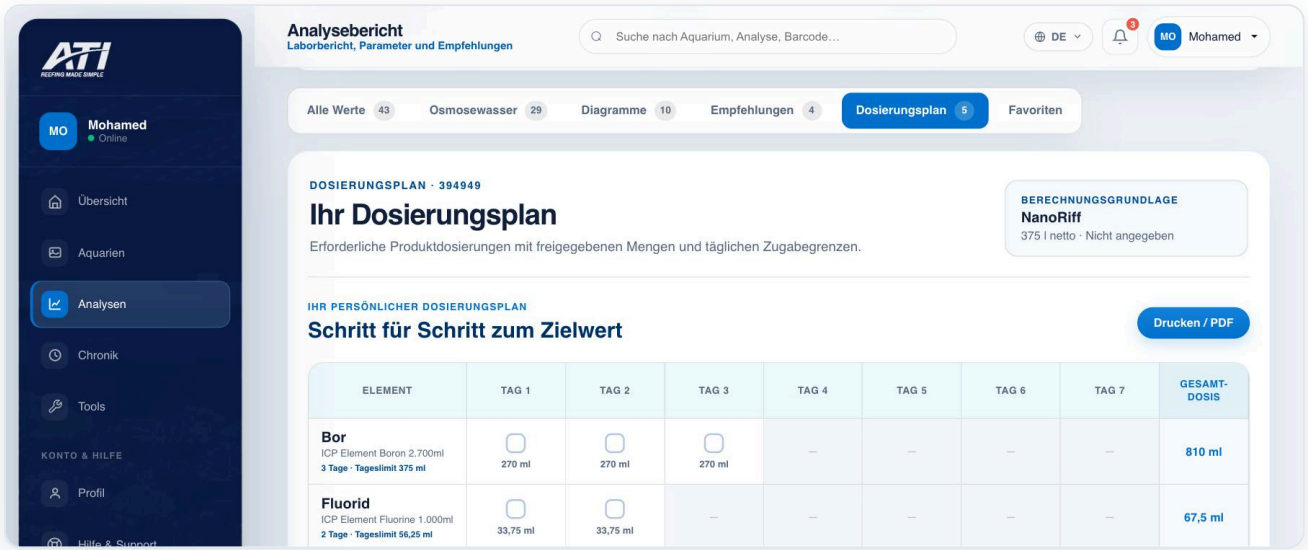
SCREENS 13–14 · TABS „EMPFEHLUNGEN“ · „DOSIERUNGSPLAN“

Recommendations and dosing plan

What the customer should do. Cards are built from the measured values, the tank's supply system and the editorial rules; the dosing plan turns deficits into millilitres per day.

● Reads & writes **T6** **T10**





EMPFEHLUNGEN

- Cards with priority, the V2.2 daily-supply adjustment (KH / Ca / Mg ± 10–20 %), tips and matching ATI or NYOS products.
- The checkbox top right ticks a card off – stored per user and report.

parameters · aquarium supply system · config: rules, templates · ticked off ✓

DOSIERUNGSPLAN

Day-by-day ml per product from deficit × tank volume ÷ product formula, capped by the daily maximum.

parameters · net_volume · config: dosing · buildDosingPlan()

CALLS TODAY → FUTURE

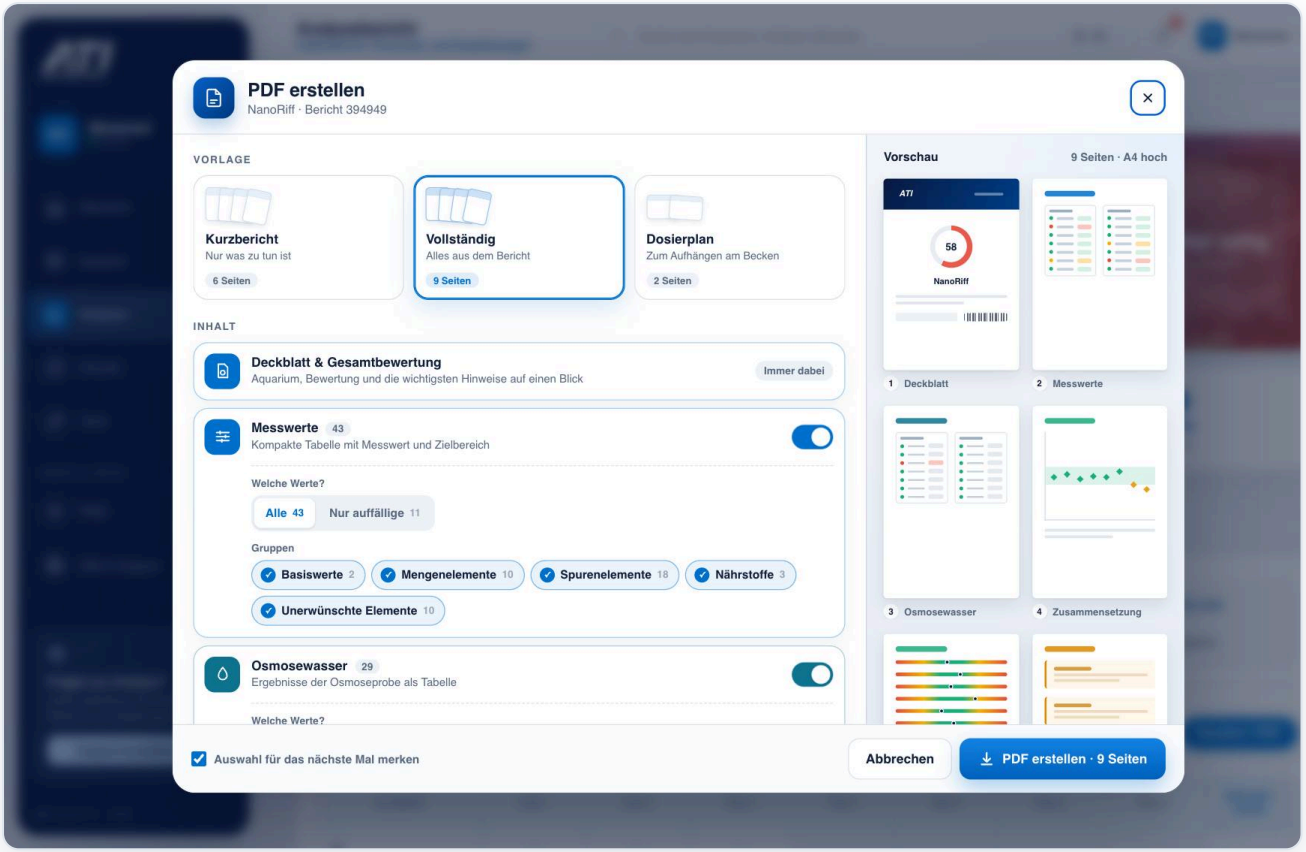
`buildDirectRecommendations()`	`stays in browser`
`loadRecommendationTemplates()`	`GET /config`
`saveRecommendationProgress()`	`PUT /me/documents/…`
`loadDosingConfig()`	`GET /config`

SCREEN 15 · „PDF ERSTELLEN“ ON THE REPORT

PDF export dialog

The dialog is finished: templates, sections with options, paper and colour, a personal note and a live page preview. What it cannot do yet is produce the file – that needs the server.

● Reads & writes T14



On „PDF erstellen“ the dialog emits `create` with `{ settings, pages }`; today `ReportView.onPdfCreate` only shows „… wird mit dem Backend verbunden.“

WHAT THE USER CHOOSES

- Template: Kurzbericht, Vollständig, Dosierplan.
- Sections: cover, values, osmosis, charts, recommendations, dosing, products (products only together with recommendations).
- Paper, colour, aquarium data, barcode, note (max. 280 characters).

WHAT GETS SENT · JSON, NO FILE

```
POST /api/analyses/394949/pdf
{ "v": 1, "settings": {
    "sections": { "cover": true, "values": true, … },
    "values": { "scope": "all", "groups": ["basis", …] },
    "format": { "orientation": "portrait", "color":
"color" },
    "note": "Für die Beratung am Freitag" } }
```

DATA

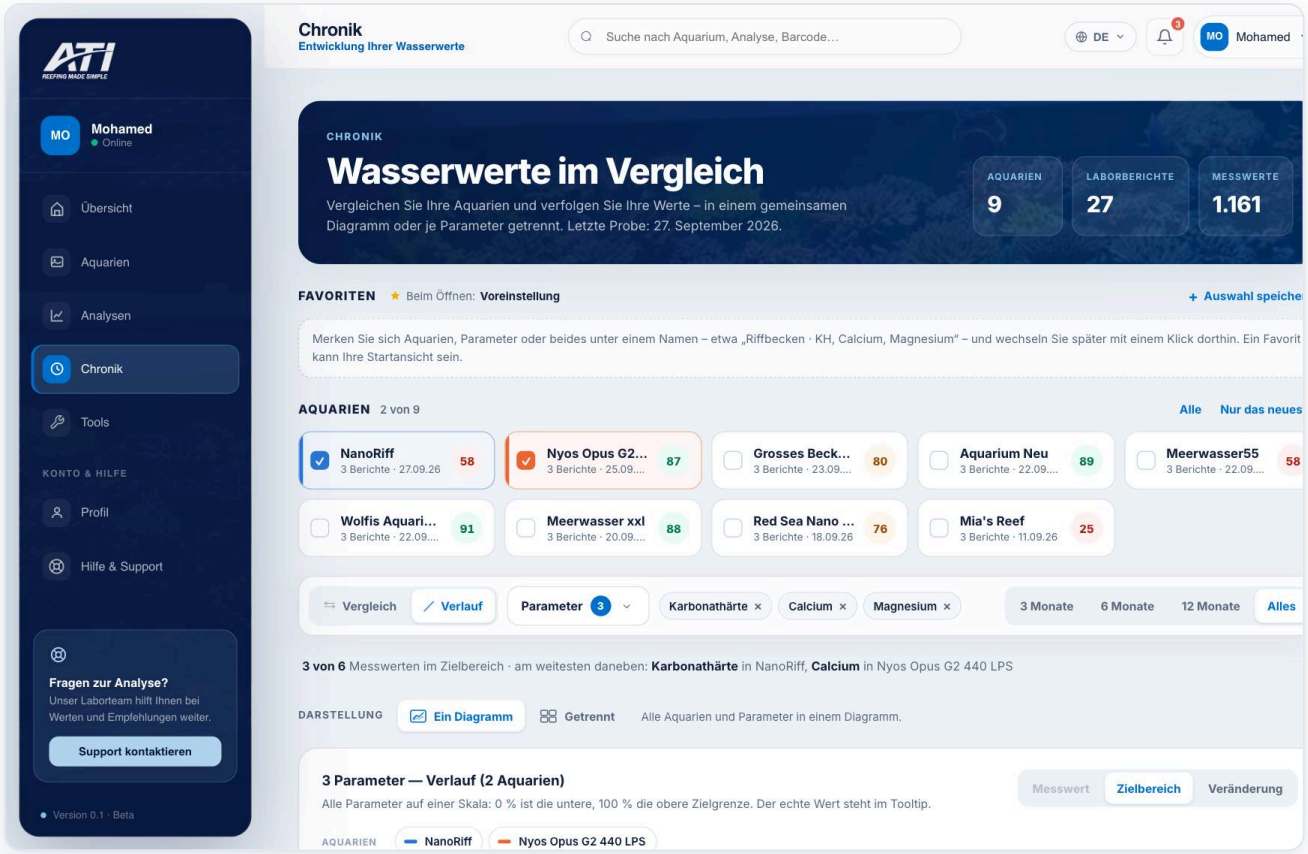
create PDF · remember choice (localStorage) · page estimate · estimatePages()

SCREEN 16 · /CHRONIK

Chronik – history across tanks

All finished reports of all tanks as lines over time. The customer picks tanks and parameters, compares or follows a trend, and saves selections as favourites.

● Reads & writes T11 T6



Built by `chronik.buildTanks(analyses, aquariums)` – it needs every completed analysis **with its parameters**, not only the list fields.

WHAT HAPPENS

- Up to six tanks get colours; more stay grey.
- Ranges: last 3 / 6 / 12 months, all.
- Favourites (max. 12) remember tanks, parameters, view and range; one can be the start view.

DATA

analyses + parameters · aquariums · config: content · favourites

CALLS TODAY → FUTURE

`analyses.load()`	`GET /analyses?include=parameters`
`loadFavorites(localStorage, …)`	`GET /me/documents`
`saveFavorites(localStorage, …)`	`PUT /me/documents/…`
`buildTanks(), chronikChart`	`stays in browser`

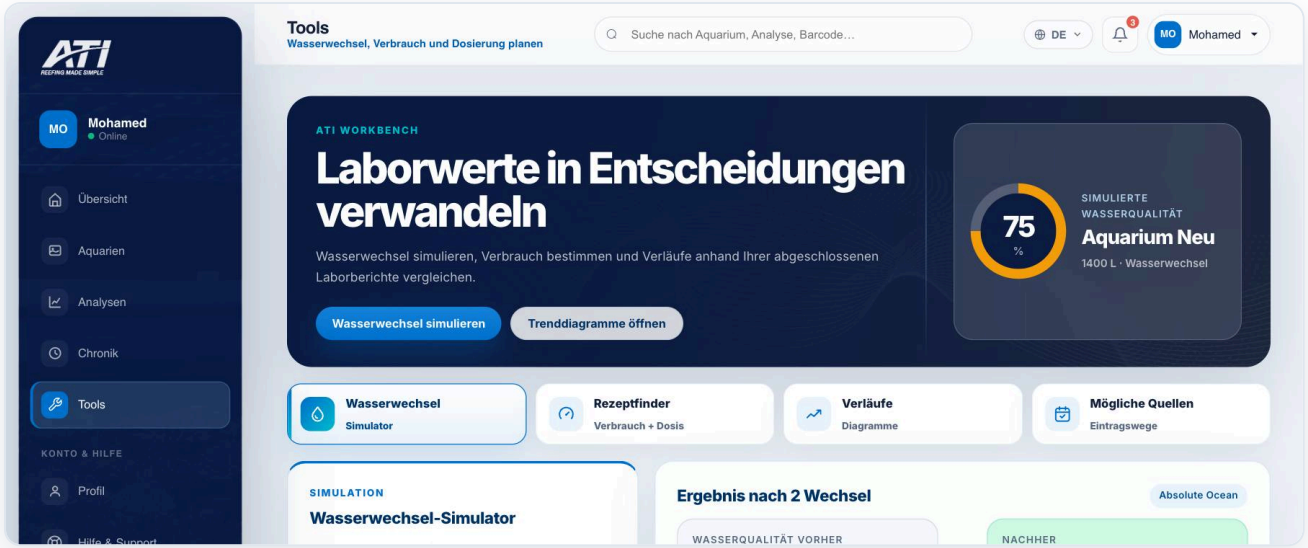
Easy win: `loadFavorites` and `saveFavorites` take a `storage` argument. Pass a server-backed storage object instead of `localStorage` and the Chronik code stays untouched (T6).

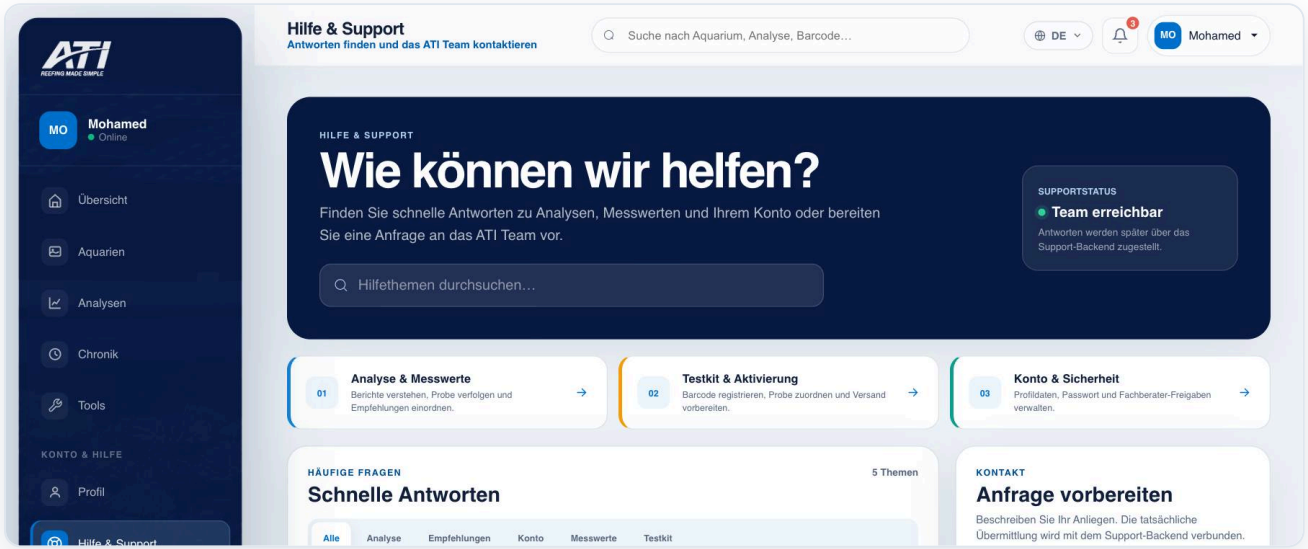
SCREENS 17–18 · /TOOLS · /SUPPORT

Tools and help

The workbench simulates water changes and consumption from finished reports; the help page shows the FAQ and lets the customer prepare a request to the ATI team.

● Reads T3 T13





TOOLS · PURE CALCULATION

Wasserwechsel-Simulator, Rezeptfinder (Verbrauch + Dosis), Verläufe, Mögliche Quellen. All maths in `toolsCalculations.js` ; data through `toolsData.js` , which already has async `profileApi` / `analysisApi` wrappers – the perfect place to call the API.

aquariums · completed analyses · simulateWaterChanges()

HILFE & SUPPORT

- FAQ with search and categories – edited by admins.
- „Anfrage vorbereiten“ only says „Die Übermittlung wird mit dem Support-Backend verbunden.“ – it needs an endpoint that e-mails the team.

config: supportFaqs · support request

CALLS TODAY → FUTURE

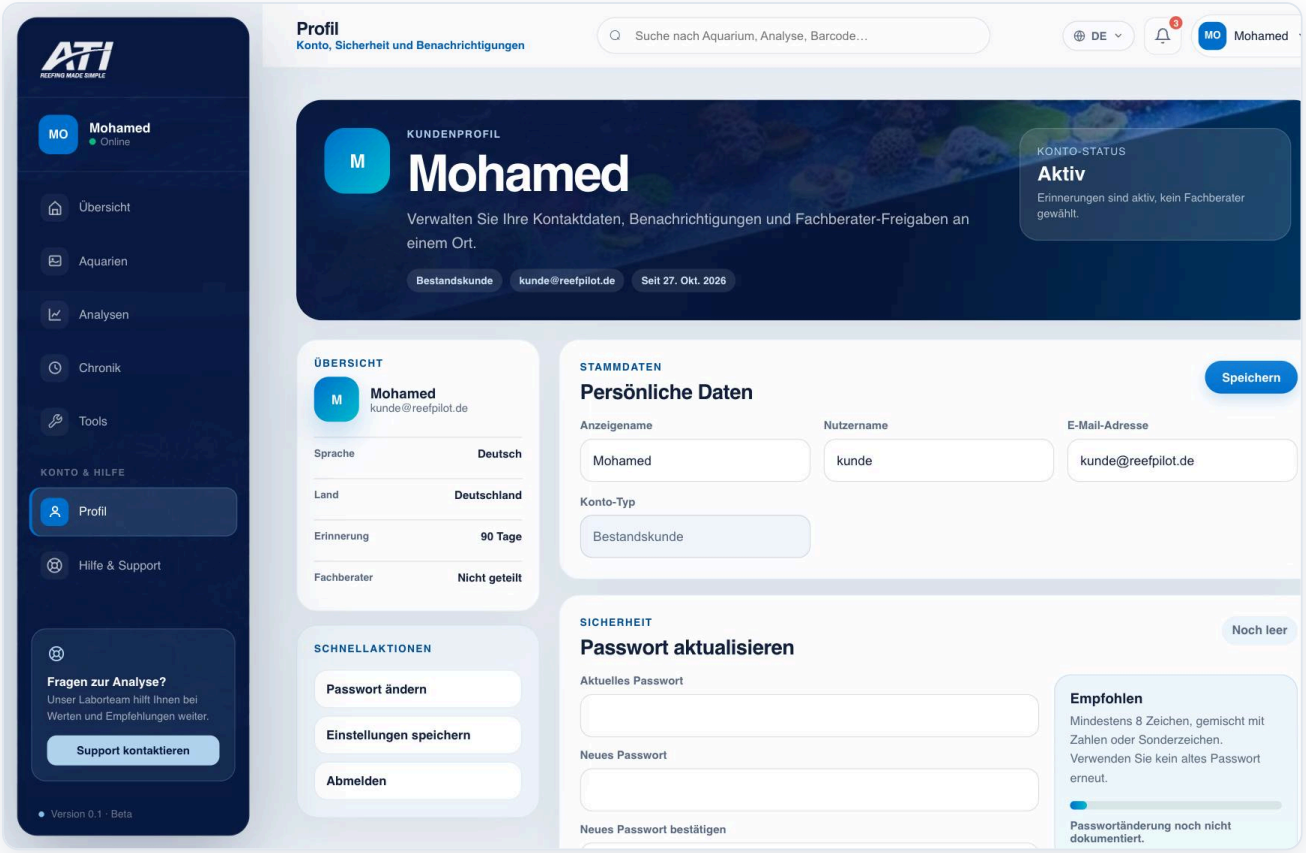
`profileApi.list()`	`GET /aquariums`
`analysisApi.list({ status })`	`GET /analyses?status=`
`loadSupportContent()`	`GET /config/supportFaqs`
`prepareRequest()`	`POST /support/requests`

SCREEN 19 · /ACCOUNT

Profile and account

Personal data, password, sharing with an advisor, notifications and local settings. All of it already goes through two async store actions – an easy migration.

Reads & writes T5



Sections: **Persönliche Daten**, **Passwort aktualisieren**, **Freigabe verwalten**, **Benachrichtigungen**, **Lokale Einstellungen**.

RULES THE SERVER MUST CHECK

- Username and e-mail required and unique (case-insensitive).
- Password: current one correct, new one ≥ 8 characters, confirmation matches.
- Reminder interval in days (default 90), newsletter opt-in, advisor id.

DATA

me · profile · password · reminders

CALLS TODAY → FUTURE

`auth.updateProfile(data)`	`PATCH /me`
`auth.changePassword(data)`	`POST /me/password`
`auth.logout()`	`POST /auth/logout`

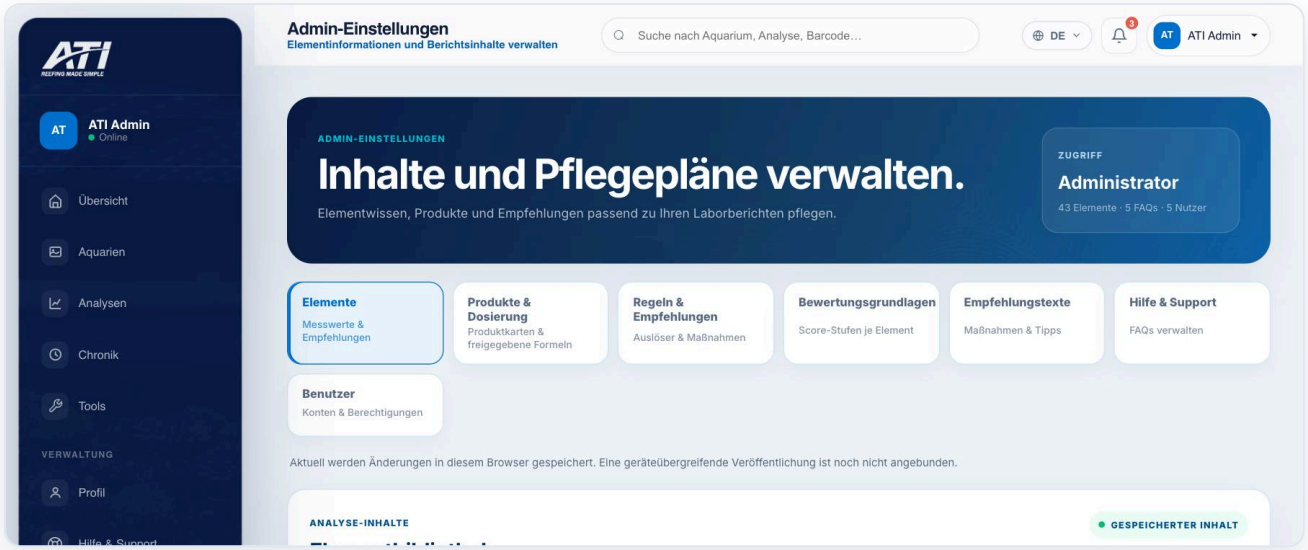
Not wired yet: `settingsPreferences.js` (units, tone of the assistant) exists but no screen imports it. Build its endpoint only when a screen uses it.

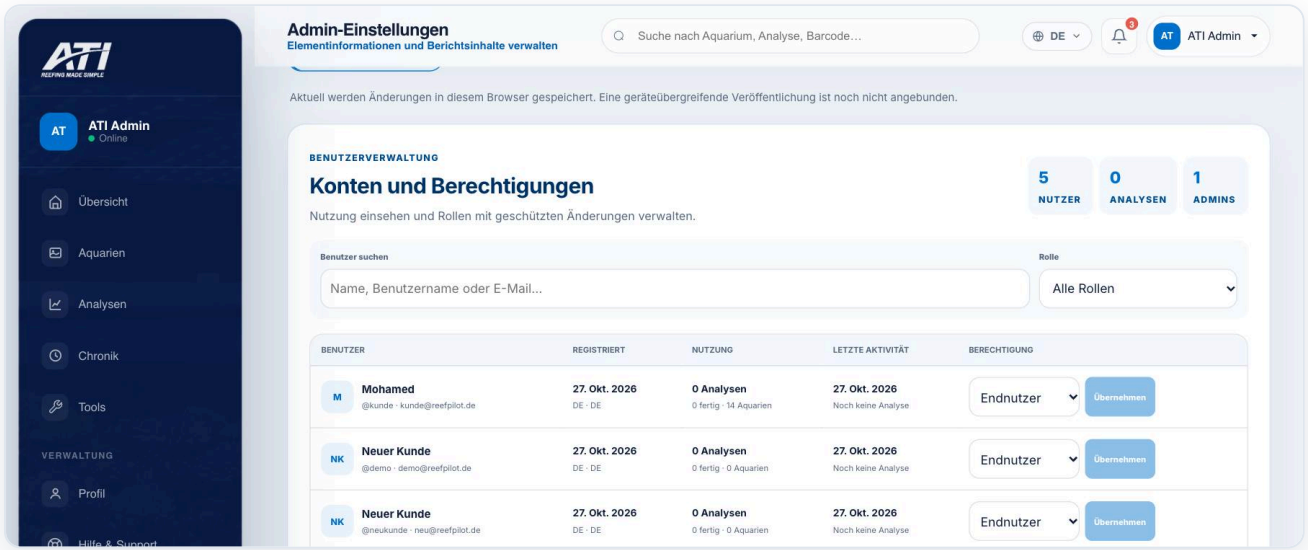
SCREENS 20–21 · /SETTINGS · ADMIN AND SUBADMIN

Editorial settings and users

The back office. Six sections define what every customer's report says; the seventh manages accounts and roles. The screen itself admits the problem: „Aktuell werden Änderungen in diesem Browser gespeichert.“

Admin T10 T7





SIX EDITORIAL SECTIONS → ONE CONFIG EACH

Elemente	`analysisContent`
Produkte & Dosierung	`dosingConfig`
Regeln & Empfehlungen	`recommendationRules`
Bewertungsgrundlagen	`evaluationScales`
Empfehlungstexte	`recommendationTemplates`
Hilfe & Support	`supportFaqs`

BENUTZER · ADMIN ONLY

List with usage counts and a role picker per user (Endnutzer, Sub-Admin, Administrator).

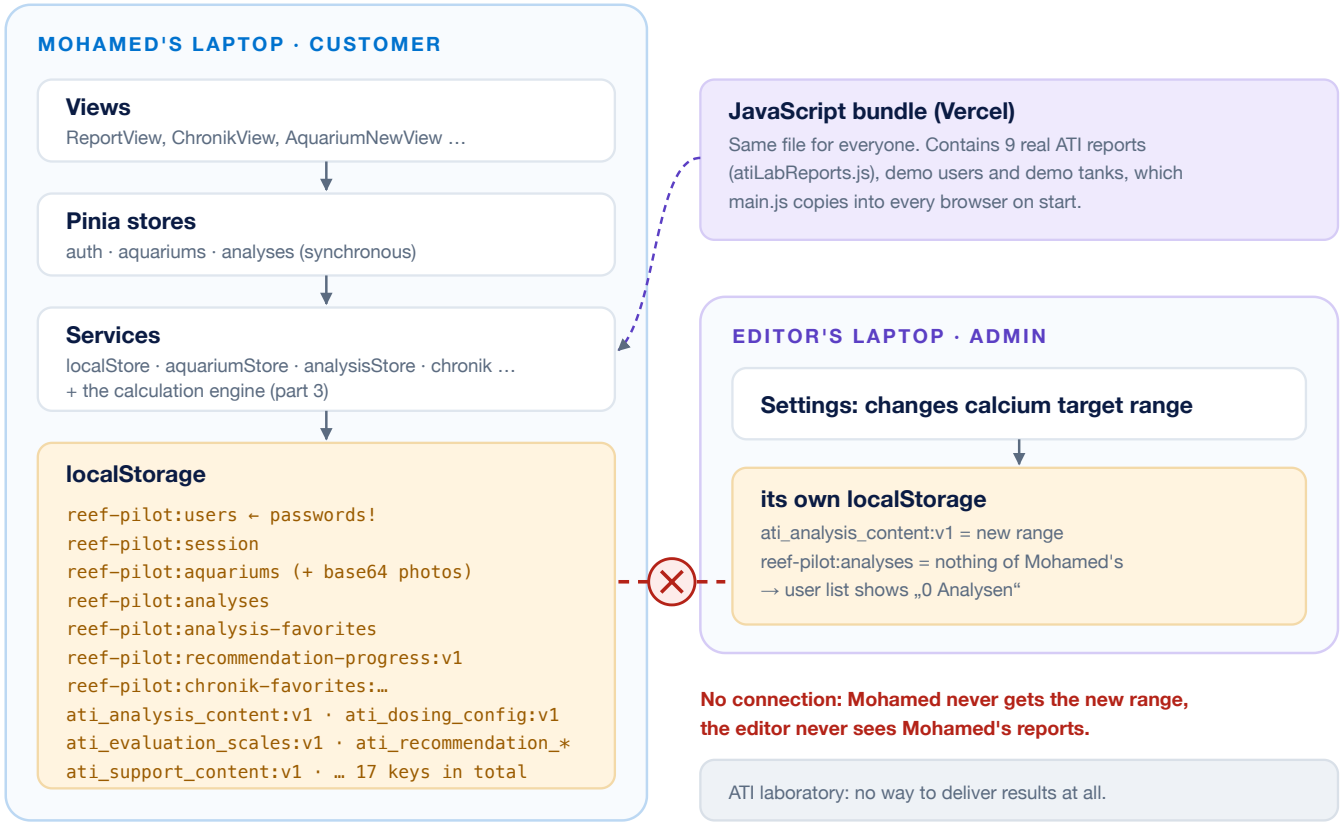
`getAdminUsers()`	`GET /admin/users`
`changeAdminUserRole()`	`PATCH /admin/users/:id/role`

Proof of the problem: the admin sees **„0 Analysen · 14 Aquarien“** for the customer Mohamed, who has nine reports. Those reports exist only in Mohamed's browser. With one shared database this number is right automatically.

PART 2 · HOW DATA MOVES

Today: every browser is its own database

There is no server. Each browser keeps a private copy of all data in `localStorage` – 17 keys. Two people never see each other's changes, and nothing survives clearing the browser.



1 · Data is per browser. A customer on a phone sees an empty account. Clearing site data deletes every aquarium.

2 · Passwords in plain text, readable by any script on the page; the "token" `local-<id>` is guessable.

3 · Editorial work stays local. Target ranges, rules, texts and FAQ changed in Settings reach only the editor.

4 · Lab results are code. The nine real reports ship inside the JavaScript bundle and belong to the demo account.

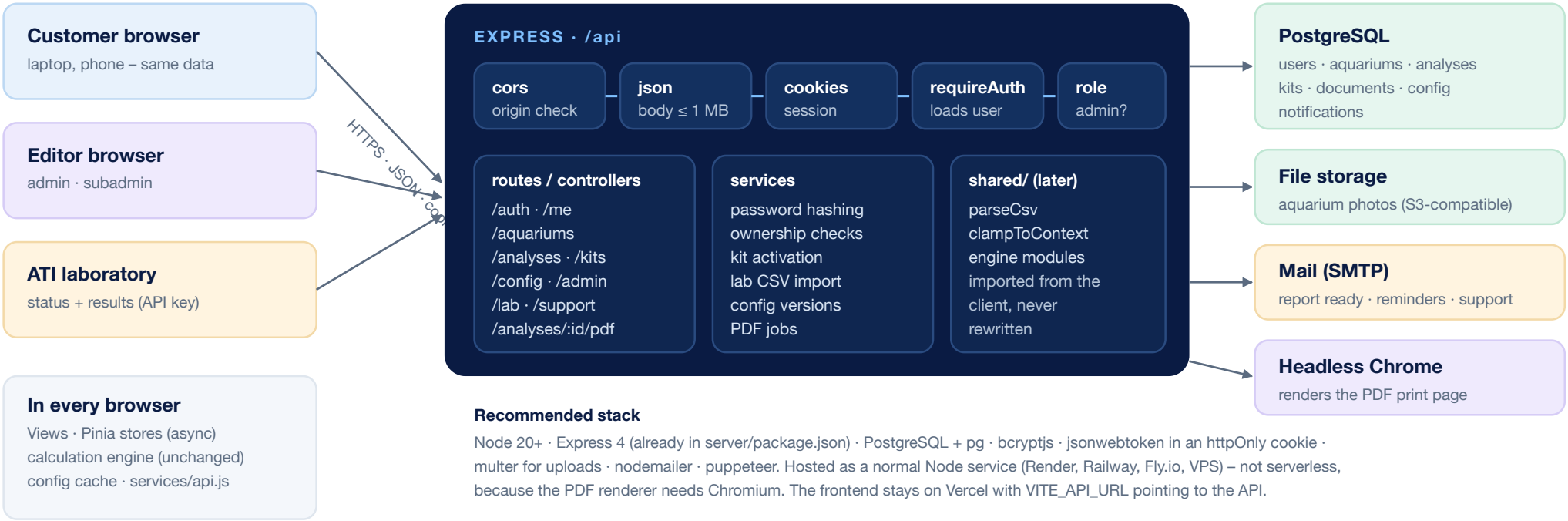
5 · Nothing arrives from outside. No lab status, no "report ready" e-mail, no support request, no PDF.

What is already good: every read and write goes through a few service files, and the views never touch `localStorage` directly – except for three small preferences.

PART 2 · HOW DATA MOVES

Tomorrow: one server, one truth

The views and the calculation engine stay as they are. Only the bottom layer changes: the service files call `/api` instead of `localStorage`. The server owns accounts, tanks, lab results and the editorial config.



Recommended stack

Node 20+ · Express 4 (already in server/package.json) · PostgreSQL + pg · bcryptjs · jsonwebtoken in an httpOnly cookie · multer for uploads · nodemailer · puppeteer. Hosted as a normal Node service (Render, Railway, Fly.io, VPS) – not serverless, because the PDF renderer needs Chromium. The frontend stays on Vercel with VITE_API_URL pointing to the API.

Views don't change. All components receive data as props. The migration happens in 3 stores and ~12 service files.

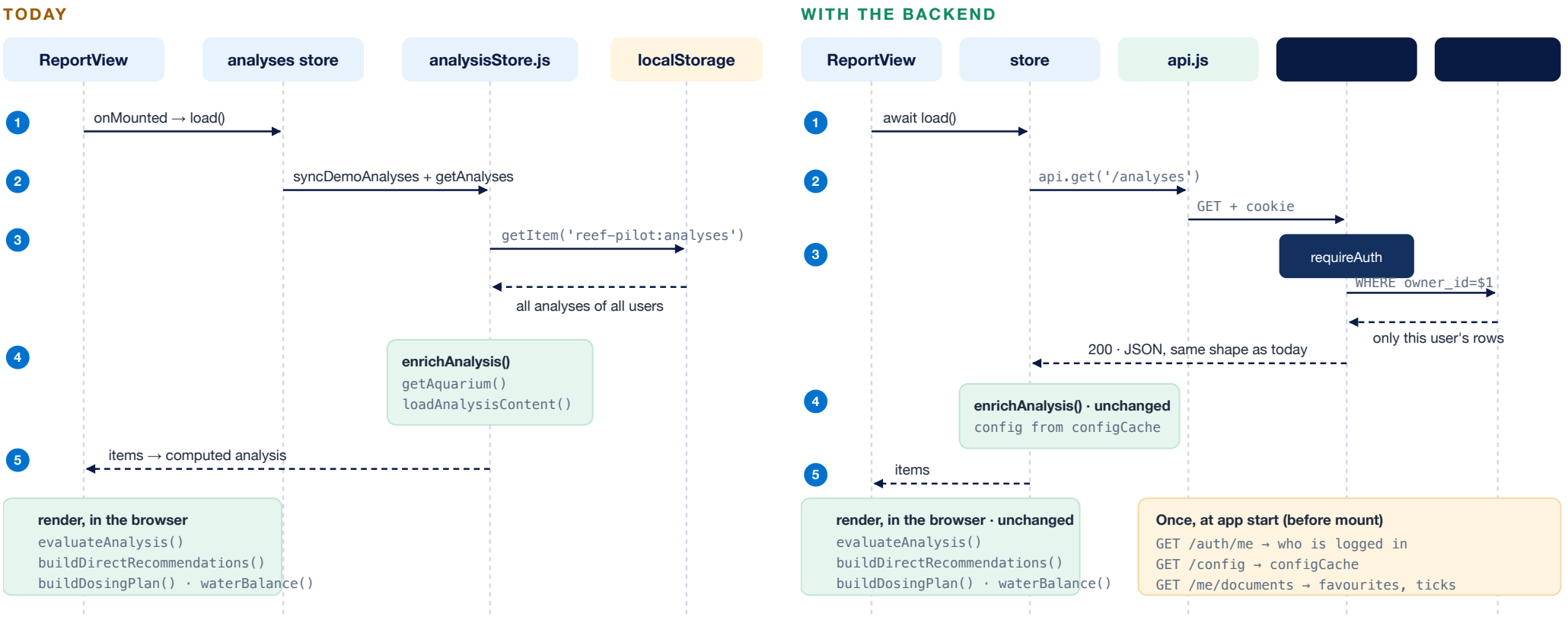
Stores become async. `aquariums.create()` today returns the tank immediately; tomorrow it returns a promise. That is the one change that touches views.

Logic stays in the browser. Scores, recommendations and dosing keep running client-side. The server stores raw data and config – no second copy of the rules.

PART 2 · ONE REQUEST, TRACED

What happens when a customer opens a report

Left: the code path today. Right: the same moment with the backend. Steps 4–6 are identical – that is the whole migration strategy in one picture.



PART 3 · BACKEND OR NOT

Which screen needs which part of the backend

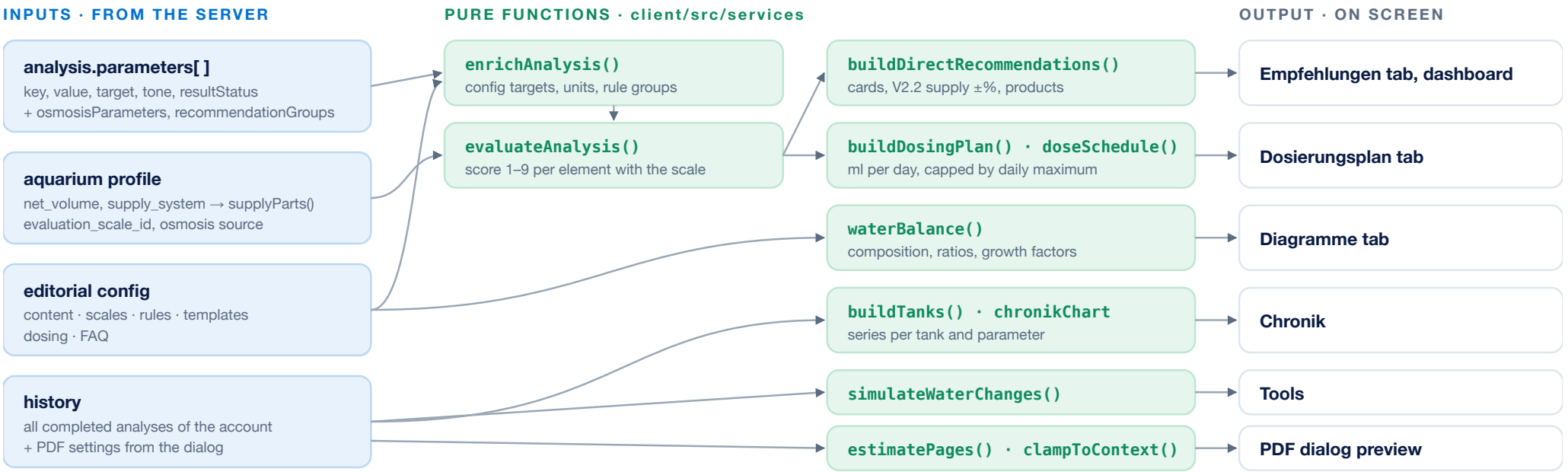
Read a row to see what a screen waits for; read a column to see which screens a task unlocks. ● reads · ◆ writes. Empty rows at the bottom of a column mean that task can be tested in isolation.

SCREEN	ROUTE / TAB	FAQ READ T3	LOGIN & SESSION T4	PROFILE & PASSWORD T5	PERSONAL LISTS T6	USER MANAGEMENT T7	AQUARIUMS T8	PHOTOS T9	EDITORIAL CONFIG T10	ANALYSES & KITS T11	LAB IMPORT T12	MAIL & NOTICES T13	PDF T14	NEEDS NOTHING –
PUBLIC														
Landing	/		●											✓
Login · Registrierung	/login · /register		◆											
CUSTOMER														
Top bar (every page)	AppLayout		●									●		
Dashboard	/dashboard		●				●	●	●	●				
Aquarien	/aquariums		●				●	●						
Neues Aquarium	/aquariums/new		●				◆	◆	●					
Aquarium-Detail	/aquariums/:id		●				◆	◆		●				
Analysen	/analyses		●							●	●			
Testkit aktivieren	/analyses/activate		●				●			◆				
Bericht · Alle Werte	tab values		●		◆		●		●	●				
Bericht · Osmosewasser	tab osmosis		●						●	●				
Bericht · Diagramme	tab balance		●							●				✓
Bericht · Empfehlungen	tab overview		●		◆		●		●	●				
Bericht · Dosierungsplan	tab dosing		●				●		●	●				
Bericht · Favoriten	tab favorites		●		◆					●				
PDF erstellen	dialog		●		◆					●			◆	
Chronik	/chronik		●		◆		●		●	●				
Tools	/tools		●				●		●	●				
Profil	/account		●	◆										
Hilfe & Support	/support	●										◆		
ADMIN														
Einstellungen (6 sections)	/settings	◆	●						◆	●				
Benutzer	/settings · Benutzer		●			◆								

PART 3 · WHAT NEVER NEEDS THE BACKEND

The calculation engine stays in the browser

Most of what the customer sees is computed, not stored. These functions take plain data in and give plain data out – no Vue, no storage. The server only has to deliver their inputs.



PURE MODULES · SAFE TO SHARE WITH THE SERVER LATER

`analysisCatalog` · `directRecommendations` · `dosingPlan` · `dosingCalendar` · `waterBalance` · `targetGauge` · `pollutantSources` · `productEligibility` · `atiProductCatalog` · `nyosProductCatalog` · `atiDosingDefaults` · `supplySystem` · `chronik` · `chronikChart` · `toolsCalculations` · `pdfExport` · `aquariumWizard` · `waterTypeFields` · `adminValidation`

Proof: their tests already run in plain Node (`npm run test:recommendations` , 131 tests).

STAYS IN THE BROWSER · NO ENDPOINT AT ALL

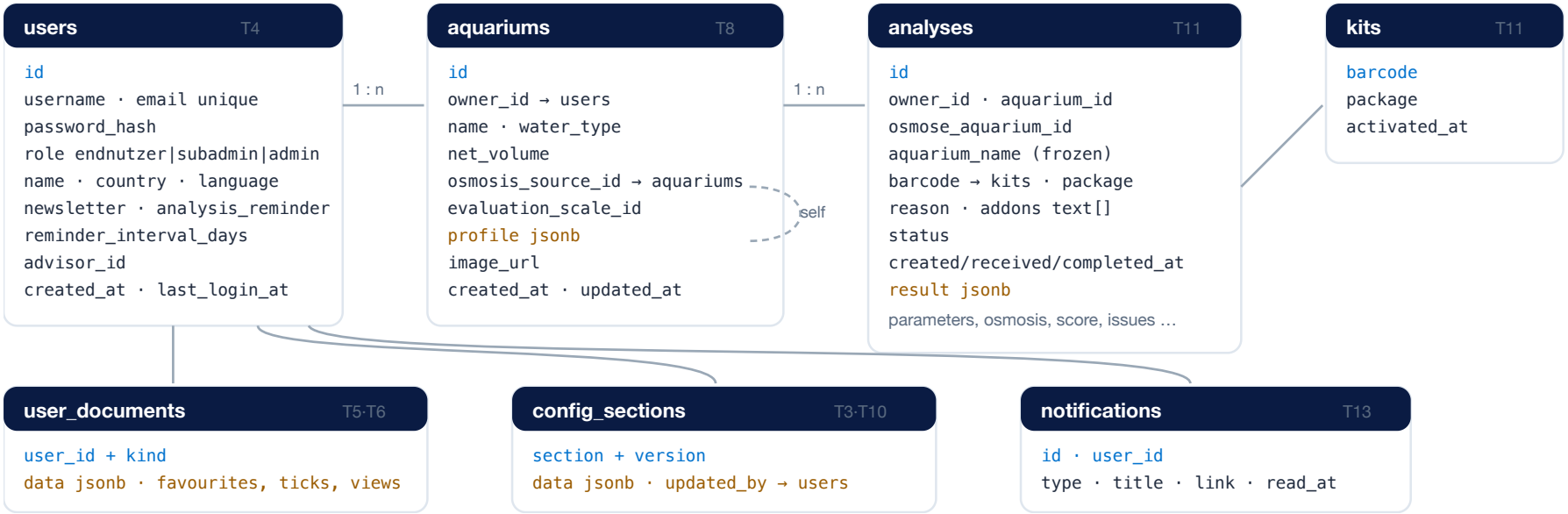
- Cards / table choice in the report (`reef-pilot:values-view`).
- The aquarium wizard draft (`aquariumWizard.saveDraft`).
- Every chart, gauge, score ring and page estimate.
- The landing page content.

One trap. Some pure functions read config through a default argument, e.g. `buildDosingPlan(parameters, volume, dosingConfig = loadDosingConfig())` and `evaluateRecommendationRules(analysis, rules = loadRecommendationRules())` . In the browser that keeps working through the config cache (T10). If the server ever calls them (PDF, e-mail), pass the config explicitly.

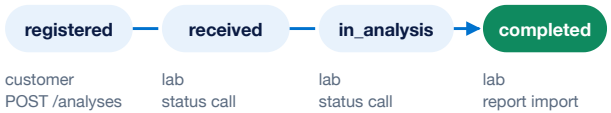
PART 3 · WHAT THE SERVER STORES

Data model

Seven tables. Field names follow what the frontend already uses, so responses map 1 : 1 onto today's objects. Flexible parts are `jsonb` – lab results, aquarium profile details and editorial config change shape often.



ANALYSIS LIFECYCLE



Only forward. Only the lab or an admin moves it.

CONFIG = DEFAULTS IN CODE + OVERRIDES IN DB

Every `load…()` starts from built-in defaults and overlays what is stored. The server stores **only what an editor changed**, in exactly the JSON the `save…()` functions write today. An empty table means "ATI defaults" – the app works from day one.

WHY `RESULT JSONB` ON ANALYSES

The frontend expects one object with `parameters`, `osmosisParameters`, `recommendationGroups`, `score` …. Storing it as one document is the fastest path. Split `parameters` into rows only if SQL queries across reports are needed later.

PART 4 · ROADMAP

14 tasks, from easiest to hardest

Each task ends with something visible in the running app. The order follows difficulty; where a task needs another one, it comes after it. Estimates are for one developer who knows Node and SQL.

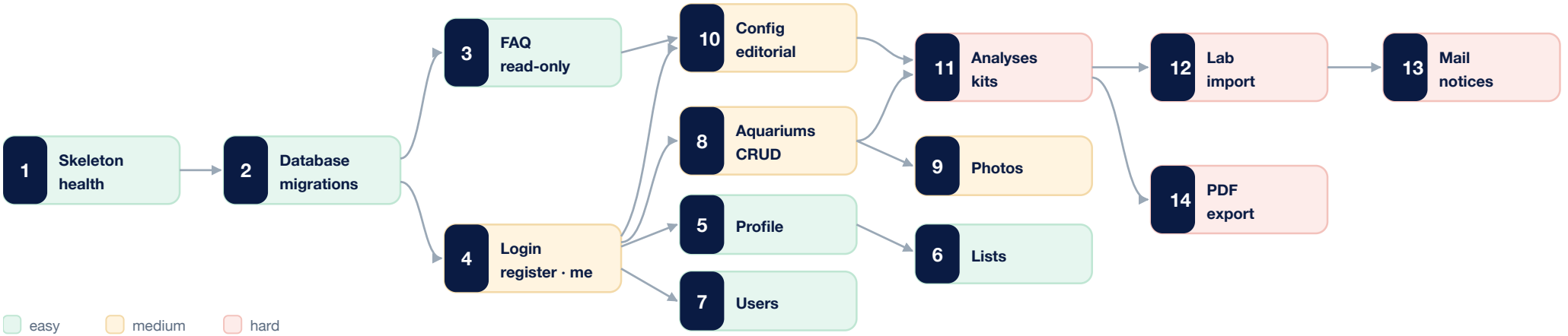
≈ 27–40
working days in total

	TASK	DIFFICULTY	EFFORT	NEEDS	UNLOCKS ON SCREEN	PAGE
	WARM-UP · INFRASTRUCTURE AND A FIRST WIN					
1	**Server skeleton & health check** Express app, error handling, dev proxy	● ○ ○ ○ ○	½ day	–	local setup works	25
2	**Database & migrations** PostgreSQL, migration runner, seed script	● ○ ○ ○ ○	1 day	T1	–	26
3	**FAQ from the server** first real data end to end, read-only	● ○ ○ ○ ○	½ day	T2	Hilfe & Support	27
	ACCOUNTS · EVERYTHING ELSE DEPENDS ON THIS					
4	**Login & registration** hashed passwords, session cookie, restore on start	● ● ● ○ ○	2–3 days	T2	Login · Registrierung · every private page	28
5	**Profile & password** profile update, password change, reminder settings	● ● ○ ○ ○	1 day	T4	Profil	29
6	**Personal lists** favourites, ticked recommendations, Chronik views, PDF choice	● ● ○ ○ ○	1–2 days	T5	★ Favoriten · Empfehlungen ✓ · Chronik	30
7	**User management** user list with usage, protected role changes	● ● ○ ○ ○	1 day	T4	Einstellungen › Benutzer	31
	CUSTOMER DATA					
8	**Aquariums** CRUD with ownership, osmosis source link	● ● ● ○ ○	2–3 days	T4	Aquarien · Neues Aquarium · Detail	32
9	**Aquarium photos** upload to file storage instead of base64	● ● ● ○ ○	1–2 days	T8	photos on all tank screens	33
10	**Editorial configuration** one config for everyone, versioned, config cache in the client	● ● ● ○ ○	3–4 days	T3, T4	Einstellungen · every report	34
	THE LAB LOOP					
11	**Analyses & kit activation** list, report, barcode check, activation	● ● ● ● ○	3–4 days	T8, T10	Analysen · Testkit · Bericht · Dashboard · Chronik · Tools	35
12	**Lab workflow & report import** status updates, ATI CSV import, fixtures	● ● ● ● ○	3–5 days	T11	lab timeline · real reports	36
13	**Notifications & e-mail** "report ready", reminders, support requests, badge	● ● ● ● ○	2–3 days	T12	top bar badge · Support form	37
14	**PDF export** print page + headless Chrome, download	● ● ● ● ●	4–6 days	T11	PDF erstellen	38

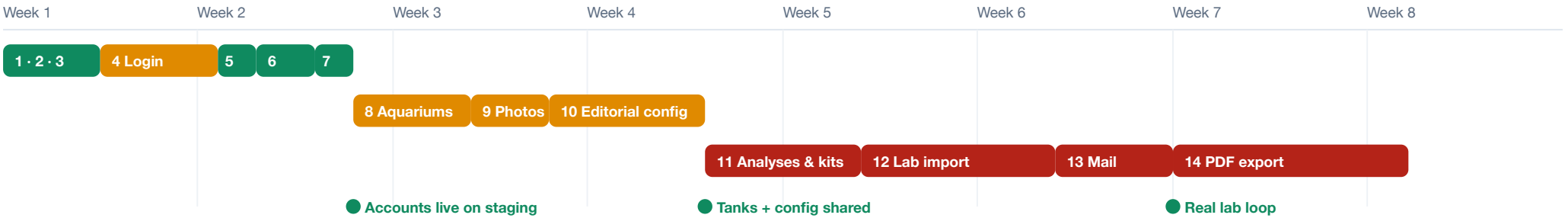
PART 4 · ROADMAP

Dependencies and a realistic timeline

Arrows mean "needs". Tasks in the same column can run in parallel with a second developer. The timeline assumes one developer and the upper estimates.



TIMELINE · ONE DEVELOPER



Ship to staging after every task. Each checkpoint can be shown to the customer – no big-bang release.

Keep the prototype working. Until T11 the analyses still come from the demo seed; nothing breaks while the rest moves.

Before T12 starts, ATI must answer how results arrive (CSV upload, LIMS export or API) – see the open decisions on the last page.

1

PLAYBOOK · WARM-UP

Server skeleton & health check

DIFFICULTY	EFFORT	NEEDS	UNLOCKS
● ○ ○ ○ ○	½ day	–	local setup

Goal: an Express app in `server/src` that starts with `npm run dev`, answers `/api/health`, and the Vite dev server forwards every `/api` call to it.

ENDPOINTS

GET	`/api/health`	→ `{ "ok": true }`
ANY	unknown path	→ `404 { "error": "Nicht gefunden" }`

STEPS

1. `cd server && npm i cookie-parser` (express, cors, dotenv are already in `package.json`).
2. Create the app with CORS, JSON body and cookies, a central error handler that answers `{ error }`, the router with `/health` and a 404 fallback, and the start file.
3. `.env`: `PORT=3000`, `CLIENT_ORIGIN=http://localhost:5173`. Add `.env` to `.gitignore`.
4. Add the dev proxy to `client/vite.config.js` and replace `request()` in `client/src/services/api.js`.

DONE WHEN

- ☐ `curl localhost:3000/api/health` → `{"ok":true}`
- ☐ In the app's console `await (await fetch('/api/health')).json()` works through the proxy (vite: `server.proxy: { '/api': 'http://localhost:3000' }`).

WATCH OUT

- Keep the error shape `{ error }` – every view already shows `e.error`.
- `credentials: true` needs an exact origin, never `*`.

2 Database & migrations

PLAYBOOK · WARM-UP

DIFFICULTY	EFFORT	NEEDS	UNLOCKS
● ○ ○ ○ ○	1 day	T1	all later tasks

Goal: a PostgreSQL connection, numbered SQL migrations that run once each, and a seed script that fills a staging database with the demo data the prototype has today.

STEPS

1. `npm i pg` ; local database `createdb reefpilot` ; `DATABASE_URL=postgres://localhost/reefpilot` in `.env` .
2. Add a database module and a migration script that runs every file in `db/migrations` once, in name order.
3. Scripts in `server/package.json` : `"migrate": "node scripts/migrate.js"` , `"seed": "node scripts/seed.js"` .
4. Make `/api/health` also run `SELECT 1` , so a broken database shows up in monitoring.
5. `scripts/seed.js` starts empty; every later task adds its part (demo users T4, demo tanks T8, reports T12). It refuses to run when `NODE_ENV=production` .

Why plain SQL files? Small, readable, no ORM to learn. Prisma, Knex or Drizzle work just as well – pick one before T4 and keep it.

DONE WHEN

- ☐ A fresh database reaches the latest schema with one command.
- ☐ Running it again changes nothing.

WATCH OUT

- Never edit a migration that is merged – add a new one.
- `DATABASE_URL` lives in `.env` , never in git.

3 FAQ from the server

PLAYBOOK · FIRST WIN

DIFFICULTY	EFFORT	NEEDS	UNLOCKS
● ○ ○ ○ ○	½ day	T2	Hilfe & Support

Goal: the first real data travels from the database to a screen. The FAQ on `/support` comes from `config_sections` . Small, public, read-only – the perfect end-to-end test of T1 + T2.

ENDPOINT

`GET`	`/api/config/supportFaqs`	public · `{ faqs: [ { id, category, question, answer } ] }`

STEPS

1. Migration `002_config.sql` .
2. Seed version 1 from the FAQs that already exist in the client (`DEFAULT_SUPPORT_FAQS` in `client/src/services/supportContent.js`).
3. Route `routes/config.js` , mounted in `routes/index.js` .
4. In `SupportView.vue` keep the built-in FAQ as first paint and replace it when the server answers.

DONE WHEN

- ☐ The FAQ comes from the database.
- ☐ The page still works with the API down.

WATCH OUT

- Only whitelisted sections are public.
- T10 replaces this one-off with the config cache – keep the code small.

4 Login & registration

PLAYBOOK · ACCOUNTS

DIFFICULTY	EFFORT	NEEDS	UNLOCKS
●●●○○	2–3 days	T2	every private page

Goal: real accounts. Passwords hashed, a session in an httpOnly cookie, the logged-in user restored on every page load – and no account data left in `localStorage`.

ENDPOINTS

POST	`/auth/register`	`{ username, email, password, country }` → user · 201
POST	`/auth/login`	`{ login, password }` – login = username **or** e-mail
POST	`/auth/logout`	clears the cookie · 204
GET	`/auth/me`	current user or 401

STEPS

1. `npm i bcryptjs jsonwebtoken` ; `JWT_SECRET` (32+ random bytes) in `.env` .
2. `middleware/auth.js` : `requireAuth` verifies the cookie and loads the user row (so role changes apply at once); `requireRole(...roles)` .
3. `publicUser(row)` returns the same fields as `localStore.publicUser` , plus `kind: 'full'` once the user has an aquarium.
4. Seed the five demo accounts with hashed `demo123` (staging only).
5. Frontend: the auth store logs in through the API, `main.js` awaits `auth.restore()` before `app.mount` and drops `ensureDemoUsers()` .

DONE WHEN

- ☐ Register in Chrome, log in with the same account in Safari.
- ☐ Reload keeps you logged in; no password or token in DevTools › Application.

WATCH OUT

- The router guard reads `isLoggedIn` synchronously – restore before mounting.
- Rate-limit login and register (go-live checklist).

5 Profile & password

PLAYBOOK · ACCOUNTS

DIFFICULTY	EFFORT	NEEDS	UNLOCKS
●●○○○	1 day	T4	Profil

Goal: everything on `/account` saves on the server: personal data, reminders, advisor sharing and the password. Plus the small `user_documents` table that T6 builds on.

ENDPOINTS

PATCH	`/me`	username, email, name, country, language, newsletter, analysis_reminder, reminder_interval_days, advisor_id
POST	`/me/password`	`{ current_password, new_password, confirm_password }`

RULES · SAME TEXTS AS LOCALSTORE.JS TODAY

username empty	„Bitte geben Sie einen Nutzernamen an.“
email empty	„Bitte geben Sie eine E-Mail-Adresse an.“
taken by someone else	„Benutzername oder E-Mail ist bereits vergeben.“
current password wrong	„Aktuelles Passwort ist falsch.“
new < 8 characters	„Das neue Passwort muss mindestens 8 Zeichen lang sein.“
confirmation differs	„Die neuen Passwörter stimmen nicht überein.“

Skip for now: `settingsPreferences.js` exists but no screen imports it. When one does, store it as `kind = 'preferences'` via T6.

DONE WHEN

- ☐ Profile changes appear on another device.
- ☐ Every rule on the left returns its German message.

WATCH OUT

- Whitelist fields – never spread `req.body` into SQL.
- After a password change, consider ending other sessions.

6 Personal lists

PLAYBOOK · ACCOUNTS

DIFFICULTY	EFFORT	NEEDS	UNLOCKS
● ● ○ ○ ○	1–2 days	T5	★ · ✓ · Chronik

Goal: the four small per-user lists follow the user to every device: favourite elements (★), ticked recommendations (✓), saved Chronik views and the remembered PDF choice. One generic endpoint, one tiny client adapter.

ENDPOINTS

GET	`/me/documents`	all at once: `{ "favorite-parameters": [...], "chronik-views": {...} }`
PUT	`/me/documents/:kind`	replace one · 204
DEL	`/me/documents/:kind`	remove one · 204

WHAT GOES WHERE

`favorite-parameters`	`analysisStore.get/saveFavoriteParameters`
`progress:<analysisId>`	`recommendationProgress.load/save…`
`chronik-views`	`chronik.loadFavorites / saveFavorites`
`pdf-settings`	`pdfExport.loadPdfSettings / savePdfSettings`

STEPS

1. Routes with a whitelist of kinds and a 64 KB size cap.
2. New `client/src/services/remoteStorage.js`; call `loadDocuments()` right after login and after `restore()`.
3. Chronik and PDF dialog: pass `remoteStorage` instead of `localStorage` – their functions already take a `storage` argument.
4. Favourites and progress: replace the two `read/write` helpers inside the module with the cache.

DONE WHEN

- ☐ ★ on the laptop shows on the phone.
- ☐ Ticked recommendations and Chronik favourites survive a new browser.

WATCH OUT

- Writes are fire-and-forget: show a small toast if a PUT fails.
- Last write wins – fine for these lists.

7 PLAYBOOK · ACCOUNTS
User management

DIFFICULTY	EFFORT	NEEDS	UNLOCKS
●●○○○	1 day	T4	Einstellungen › Benutzer

Goal: admins see every account with real usage numbers and change roles safely. This alone fixes the „0 Analysen“ shown for customers today.

ENDPOINTS · ROLE ADMIN ONLY

GET	`/admin/users`	users + `aquariumCount` , `analysisCount` , `completedAnalysisCount` , `lastAnalysisAt` , `lastActivityAt`
PATCH	`/admin/users/:id/role`	`{ role }` → the updated list

RULES · FROM LOCALSTORE.UPDATEUSERROLE

role not in the list	„Diese Rolle ist nicht zulässig.“
caller is not admin	„Nur Administratoren dürfen Rollen ändern.“
own account	„Die eigene Rolle kann nicht geändert werden.“
last admin demoted	„Der letzte Administrator kann nicht herabgestuft werden.“

DONE WHEN

- ☐ A promoted user sees Einstellungen right away (requireAuth reloads the role).
- ☐ Mohamed's row shows his real numbers.

WATCH OUT

- The last-admin check must run in the same transaction as the update.
- Log who changed which role (audit).

---

8 PLAYBOOK · CUSTOMER DATA
Aquariums

DIFFICULTY	EFFORT	NEEDS	UNLOCKS
●●●○○	2–3 days	T4	Aquarien · Wizard · Detail

Goal: tanks and osmosis sources live in the database, belong to one user, and come back in exactly the shape `emptyAquarium()` defines – so no view has to change its field names.

ENDPOINTS

GET	`/aquariums`	the caller's tanks, oldest first
POST	`/aquariums`	body = wizard form → created tank · 201
GET	`/aquariums/:id`	404 if not the caller's
PATCH	`/aquariums/:id`	partial update, sets `updated_at`
DEL	`/aquariums/:id`	see "Watch out"

STEPS

1. Migration and routes; the API answers in the same shape the client stores today.
2. Store: `load` , `create` , `update` , `remove` become async and call the API.
3. `aquariumStore.getAquarium(id)` is also used synchronously by `analysisStore.enrichAnalysis` and `OsmosisSourcePicker` – keep it synchronous by reading a module cache the store fills.
4. `await` in `AquariumNewView` , `AquariumDetailView` , `ActivateView` ; drop `ensureDemoAquariums()` ; seed the 14 demo tanks on staging.

DONE WHEN

- ☐ A tank made in the wizard appears on the phone.
- ☐ Another user gets 404 for its id.

WATCH OUT

- Until T9, drop `image` from the body – base64 does not belong in JSON.
- Deleting a tank with reports: decision needed (last page).

9 Aquarium photos

PLAYBOOK · CUSTOMER DATA

DIFFICULTY	EFFORT	NEEDS	UNLOCKS
●●●○○	1–2 days	T8	photos everywhere

Goal: tank photos are files in storage, not base64 text in `localStorage`. The browser keeps resizing them to 900 px (`imageUtil.fileToResizedDataUrl`) and uploads the result.

ENDPOINTS

POST	`/aquariums/:id/image`	multipart, field `image` , JPEG/PNG/WebP ≤ 5 MB → `{ image: url }`
DEL	`/aquariums/:id/image`	back to the illustration (`image_theme`)

HOW A PHOTO TRAVELS



The view shows the returned URL exactly where it shows the data URL today.

aquariums.image_url = public URL
old file is deleted

STEPS

1. `npm i multer` (+ `@aws-sdk/client-s3` for production).
2. A tiny `storage.js` with `put(key, buffer, type)` / `remove(key)` : local folder in development, S3-compatible bucket in production.
3. Add `api.upload(path, formData)` to `api.js` – it must **not** set `Content-Type` (the browser adds the boundary).
4. Wizard: create the tank first (T8), then upload the photo with the new id.

DONE WHEN

- ☐ Photos show on every device and survive clearing the browser.
- ☐ `localStorage` holds no image data any more.

WATCH OUT

- Check the real file type (magic bytes), not only the header.
- Use unguessable keys; strip EXIF location data.

10

PLAYBOOK · CUSTOMER DATA

Editorial configuration

DIFFICULTY	EFFORT	NEEDS	UNLOCKS
●●●○○	3–4 days	T3 · T4	Einstellungen · every report

Goal: what an editor saves in Settings reaches every customer. The trick: the seven `load…()` / `save…()` pairs keep their code – only the word `localStorage` becomes `configStorage`, a cache filled once after login.

ENDPOINTS

GET	`/config`	logged in · `{ sections: { name: data | null } }`
PUT	`/admin/config/:section`	admin, subadmin · `{ value }` → new version
DEL	`/admin/config/:section`	reset to built-in defaults (new version with `null`)

STEPS

1. Routes; `config_sections` exists since T3.
2. New `client/src/services/configCache.js`; `await loadConfig()` after `restore()`.
3. In the seven files replace `localStorage` with `configStorage` (≈ 15 lines). Undo the one-off from T3 in `SupportView`.
4. `AdminSettingsView`: await the save and show an error if the server refuses.

DONE WHEN

- ☐ A sub-admin changes the calcium range; a customer on another device sees it after reload.
- ☐ A customer's `PUT` gets 403.

WATCH OUT

- The client sanitises before saving; the server still checks section, role and size.
- Keep the built-in defaults in code – an empty table must mean "ATI standard".

SEVEN SECTIONS · EACH IS ONE BROWSER KEY TODAY

`analysisContent`	`ati_analysis_content:v1`	analysisContent.js
`evaluationScales`	`ati_evaluation_scales:v1`	evaluationScales.js
`activeScaleId`	`ati_evaluation_scale_active:v1`	
`recommendationRules`	`ati_recommendation_rules:v1`	recommendationRules.js
`recommendationTemplates`	`ati_recommendation_templates:v1`	recommendationTemplates.js
`dosingConfig`	`ati_dosing_config:v1`	dosingConfig.js
`supportFaqs`	`ati_support_content:v1`	supportContent.js

11 PLAYBOOK · THE LAB LOOP
Analyses & kit activation

DIFFICULTY	EFFORT	NEEDS	UNLOCKS
●●●●○	3–4 days	T8 · T10	Analysen · Bericht · Chronik

Goal: a customer registers a kit, sees it in the list, and opens finished reports – all from the server. The report screens keep computing everything themselves through the unchanged `enrichAnalysis()`.

ENDPOINTS

GET	`/analyses`	own, newest first, full records · `?aquariumId=` `?status=`
GET	`/analyses/:id`	one record · 404 if foreign
GET	`/kits/:barcode`	`{ package, available }` – wizard step 1
POST	`/analyses`	`{ barcode, aquariumId, osmoseAquariumId, reason, addons }`

STEPS

1. Export `enrichAnalysis` from `analysisStore.js`; the store maps API records through it.
2. Load aquariums **before** analyses – `enrichAnalysis` looks the tank up synchronously.
3. Wizard step 1 calls `GET /kits/:barcode` (debounced) instead of `packageForBarcode()`.
4. Remove `syncDemoAnalyses`; T12 seeds the nine real reports.

DONE WHEN

- ☐ Kit `ATI-S-2026-0001` activates once, the second try says „bereits registriert“.
- ☐ Report 394463 looks identical to the prototype (screenshot compare).

WATCH OUT

- The list returns full records because Chronik needs `parameters` – add a summary mode when accounts grow.
- Never return another owner's rows.

12

PLAYBOOK · THE LAB LOOP

Lab workflow & report import

DIFFICULTY	EFFORT	NEEDS	UNLOCKS
●●●●○	3–5 days	T11	real reports · timeline

Goal: the laboratory moves a sample through *received* → *in analysis* and delivers the finished report. The CSV parser that already turns ATI exports into report rows moves to code both sides share – it is never rewritten.

ENDPOINTS · LAB KEY OR ADMIN

POST	`/lab/samples/:barcode/status`	`{ status: 'received' | 'in_analysis' }`
POST	`/lab/reports`	multipart: `csv` file + `meta` JSON → `completed`

WHAT THE LAB SENDS

```
# csv — the ATI export, 7 columns, decimal comma
"Calcium";"Ca";"480,00";"414,53";"+65,47";"mg/l";"+"
"Zinc";"Zn";"0,00";"1,97";"0,00";"ug/l";"n.n."
# meta — what the CSV cannot express
{ "barcode": "TK3D-EJBL-UAG9-L4BV", "reportNumber": "394527",
  "score": 82, "resultLevel": "watch", "groupScores": { … },
  "issues": ["Calcium über Soll", …],
  "belowNormal": { "zinc": "--", "manganese": "-" },
  "osmosis": { "silicon": 540.8, "copper": 5.42 },
  "recommendationGroups": [ … ] }
```

STEPS

1. Move `parseCsv`, `labParameter`, `osmosisRows`, `OSMOSIS_KEYS` from `atiLabReports.js` into a pure `atiCsv.js` and export them (later `shared/`).
2. `requireLab`: header `X-Lab-Key` compared with `crypto.timingSafeEqual`, or an admin session.
3. Seed (staging): the nine reports from `createAtiLabReports()` + `withDemoHistory()` for the demo customer, with matching kits.
4. Keep `atiLabReports.test.js` green against the moved parser.

DONE WHEN

- ☐ The customer's timeline moves step by step.
- ☐ An imported CSV renders the same report as the hard-coded one today.

WATCH OUT

- „n.n." rows need `belowNormal` from the lab; ideal values below 0,01 are rounded away (`IDEAL_BELOW_CSV_PRECISION`).
- Decide with ATI how results arrive before building the upload UI.

13 Notifications & e-mail

PLAYBOOK · THE LAB LOOP

DIFFICULTY	EFFORT	NEEDS	UNLOCKS
●●●●○	2–3 days	T12	badge · Support form

Goal: the portal reaches out. "Ihr Bericht ist fertig" when the lab finishes, the analysis reminder the profile promises, the support request from `/support` – and the fixed „3“ in the top bar becomes a real count.

ENDPOINTS

GET	`/me/notifications`	latest 30 + `unread` count
POST	`/me/notifications/:id/read`	mark one · `/read-all` for all
POST	`/support/requests`	`{ topic, message }` → mail to the ATI team

THREE TRIGGERS

Report ready	called by the lab import (T12)	notice + mail with link
Reminder	daily job, 07:00	if `analysis_reminder` and the last report is older than `reminder_interval_days`
Support	form on `/support`	mail to the team, copy to the user

DONE WHEN

- ☐ Importing a report sends the customer a mail with a working link.
- ☐ The badge shows the real unread count.

WATCH OUT

- A mail failure never fails the import or the request.
- SPF/DKIM for the sender domain; newsletter consent is separate.
- Password reset needs a new screen – plan it here.

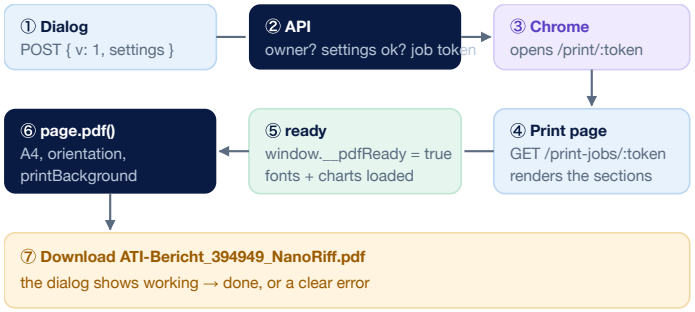
14 PDF export

PLAYBOOK · THE FINISH

DIFFICULTY	EFFORT	NEEDS	UNLOCKS
●●●●●	4–6 days	T11	PDF erstellen

Goal: „PDF erstellen“ downloads a real A4 file. Headless Chrome opens a print page of our own frontend, which renders the chosen sections with the same components the customer sees – no second layout, no second set of rules.

THE FLOW



STEPS

1. Frontend: route `/print/:token` (public, no shell) → `PrintReportView`: fetch the job, fill config cache, `enrichAnalysis`, run `clampToContext` with the real context, render the sections, set `window.__pdfReady`.
2. Values and osmosis print as the two-column compact table; products only with recommendations.
3. Server: `npm i puppeteer`; one shared browser, max. 2 pages at a time.
4. Dialog: give it back an async `printer` prop and await it (status working → done / error).

DONE WHEN

- ☐ Each template downloads; page count matches the dialog's preview.
- ☐ Unknown sections or a foreign report id are refused.

WATCH OUT

- Hosting must allow Chromium (≈ 300 MB).
- Queue renders; cache by report + settings + config version.

BEFORE REAL CUSTOMERS

Go-live checklist and open decisions

The checklist turns staging into production. The decisions are questions the code cannot answer – each needs an owner before the task named next to it.

SECURITY

- ☐ Passwords hashed, nothing secret in `localStorage`
- ☐ Cookie httpOnly · Secure · SameSite
- ☐ Rate limit on login, register, password
- ☐ Ownership tested with a 2nd account
- ☐ CORS only for the real domain
- ☐ Upload type and size checks

FRONTEND

- ☐ Preview routes → `auth: true` (router „TEMP“)
- ☐ Guest redirect back on
- ☐ Demo seeding removed from `main.js`
- ☐ `/backups/report` , `/chronik/v1` removed
- ☐ `VITE_API_URL` set on Vercel
- ☐ Loading + error state on every waiting screen

DATA

- ☐ Demo users, tanks, history only on staging
- ☐ Daily backups, one restore tested
- ☐ Config versions with author
- ☐ Account deletion and export (GDPR)

QUALITY

- ☐ 131 client tests green
- ☐ API tests for every rule on the playbooks
- ☐ Report 394463 identical to the prototype
- ☐ Monitoring + error reporting

#	DECISION	OPTIONS / WHY IT MATTERS	BEFORE
1	**Where the API runs**	Needs Chromium for PDF → Node service (Render, Railway, Fly.io, VPS), not serverless.	T1
2	**ORM or plain SQL**	Plain SQL as in this playbook, or Prisma / Knex / Drizzle – pick once.	T2
3	**File storage**	S3-compatible bucket, Vercel Blob, or the database.	T9
4	**Deleting a tank with reports**	Block it, cascade, or keep reports with the frozen tank name.	T8
5	**Which kits exist**	Server knows every issued barcode (safe), or any code with a valid prefix (simple).	T11
6	**How lab results arrive**	CSV upload by lab staff, LIMS export, or an API call. Osmosis values, „n.n.“ ratings and lab recommendations are not in today's CSV. Who sets "received"?	T12
7	**Mail provider**	SMTP of the ATI domain or a service (Postmark, SES) – needs SPF/DKIM.	T13
8	**Open V2.2 rules**	3-part system with low Ca, 2-part Ca/Mg follows Ca only, no supply system set, rating 6 on imported reports. Changes calculation code, not the API.	any
9	**Advisor sharing**	„Fachberater teilen“ and `advisor_id` exist, but no flow uses them yet.	later